



УНИВЕРЗИТЕТ У НИШУ
ЕЛЕКТРОНСКИ ФАКУЛТЕТ



Развој система за подршку раду у апотеци применом Retrieval-Augmented Generation система

Завршни рад

Студијски програм: електротехника и рачунарство

Модул: рачунарство и информатика

Студент:

Јордан Динић, бр. инд. 18625

Ментор:

Доц. др Милена Б. Фртунић Глигоријевић

Ниш, октобар 2026. година

Универзитет у Нишу
Електронски Факултет

Развој система за подршку раду у апотеци применом Retrieval-Augmented Generation система

Development of a Pharmacy Support System Using Retrieval-Augmented Generation

Завршни рад
Студијски програм: електротехника и рачунарство
Модул: рачунарство и информатика

Студент: Јордан Динић, бр. инд. 18625

Ментор: Доц. др Милена Б. Фртунић

Глигоријевић

Задатак: Упознати се са појмом и основним концептима Retrieval-Augmented Generation (RAG) система. Упознати се са основним концептима великих мултијезичних модела за векторизацију текста. Истражити основне карактеристике ChromaDB векторске базе података. У практичном делу рада, коришћењем ChromaDB базе података и великих језичких модела имплементирати систем за подршку раду у апотеци применом Retrieval-Augmented Generation sistema.

Датум пријаве рада: _____

Датум предаје рада: _____

Датум одбране рада: _____

Комисија за оцену и одбрану:

1. Доц. др Милена Б. Фртунић Глигоријевић, Председник
Комисије

2. <Проф. / Доц. др Име Презиме>, Члан

3. <Проф. / Доц. др Име Презиме>, Члан

Развој система за подршку раду у апотеци применом Retrieval-Augmented Generation система

САЖЕТАК

У раду је развијен и анализиран прототип система за подршку раду у апотеци заснован на генерисању уз помоћ претраживања, односно Retrieval-Augmented Generation приступу. Намена система је проналажење релевантних делова документације и састављање одговора на питање корисника, уз приказ пронађених извора. База знања обухвата PDF упутства за лекове, водич добре апотекарске праксе и документ о интеракцијама лекова.

Реализовано решење раздваја читавање страница, препознавање структуре докумената, сегментацију, складиштење и одговарање на питања. За представљање текста и метаподатака користи се *LangChain Document*. Упутства за лекове деле се према шест главних секција, водич према расположивим PDF обележивачима, а документ о интеракцијама према посебним правилима за наслове. Имплементиран је родитељ-дете (енг. *parent-child*) приступ: родитељски (енг. *parent*) сегменти служе за дељење PDF докумената на секције, док мањи (енг. *child*) сегменти служе за претрагу и формирање контекста у промпта. Назив лека, активна супстанца, назив PDF-а и секција укључују се у садржај сегмената када су одговарајући подаци доступни.

Chroma Cloud обавља складиштење и претрагу уз густе *Qwen* и ретке *SPLADE* векторске репрезентације. Рангови се комбинују методом *Reciprocal Rank Fusion*. Одговор генерише *Gemini*, а *Streamlit* интерфејс приказује одговоре, изворе и пронађени контекст. Историја тих података чува се током корисничке сесије, без прослеђивања претходног разговора моделу.

Локални корпус садржи 31 PDF са 339 страница, из којих настају 493 родитељске секције и 1.361 мањи сегмент. У почетној евалуацији са 15 питања, подразумевана хибридна конфигурација постиже макро *Recall@5* од 0,8667 и *MRR@5* од 0,8333, при чему се релевантност оцењује углавном према очекиваном документу и секцији. Проверен је пролазак 31 софтверског теста. Анализа издваја ограничења у раду са ћириличним називом лека, издвајању садржајних секција водича, преношењу доказа у ограничени контекст и прецизности навођења извора. Резултати потврђују функционалност прототипа, али не представљају клиничку валидацију. Систем је намењен подршци стручној процени, а не њеној замени.

Кључне речи: *RAG*, подршка раду у апотеци, мултијезичка векторизација, *ChromaDB*, хијерархијска сегментација, евалуација претраге.

Development of a Pharmacy Support System Using Retrieval-Augmented Generation

ABSTRACT

This thesis presents the development and analysis of a Retrieval-Augmented Generation prototype for pharmacy support. The system retrieves relevant passages from a document collection and generates an answer to the user's question while displaying retrieved sources. Its knowledge base consists of PDF medicine leaflets, a good pharmacy practice guide, and a reference document on drug interactions.

The implemented solution separates page loading, document structure recognition, segmentation, storage, and question answering. The LangChain Document object is used to represent text and metadata. Medicine leaflets are divided according to six main sections, the guide is segmented according to the available PDF bookmarks, and the drug-interaction document is processed according to dedicated title-detection rules. A parent-child approach is implemented: parent segments are used to divide PDF documents into sections, while child segments are used for retrieval and for forming the context included in the prompt. The medicine name, active substance, PDF filename, and section are included in the segment content whenever the corresponding data are available.

Chroma Cloud stores the records and supports search using dense Qwen and sparse SPLADE representations combined through Reciprocal Rank Fusion. Gemini generates the final answer. A Streamlit interface displays answers, retrieved sources, and context, retaining this information within the current session. Previous conversation turns are not supplied to the language model.

The local collection contains 31 PDFs with 339 pages, producing 493 parent sections and 1,361 child chunks. An initial evaluation with 15 questions yielded a macro Recall@5 of 0.8667 and MRR@5 of 0.8333 for the default hybrid configuration. Relevance was primarily assessed against expected document and section metadata. All 31 software tests passed. Identified limitations include a failed Cyrillic medicine-name query, insufficiently informative guide sections, evidence loss under the context budget, and incomplete source attribution. These results demonstrate a functional prototype rather than clinical validity. The system is intended to assist, not replace, the professional judgment of a pharmacist or physician.

Keywords: Retrieval-Augmented Generation, pharmacy support, multilingual embeddings, ChromaDB, hierarchical chunking, retrieval evaluation.

СПИСАК КОРИШЋЕНИХ СКРАЋЕНИЦА И ОЗНАКА

Скраћеница	Значење
API	Application Programming Interface, програмски интерфејс
CLI	Command-Line Interface, интерфејс командне линије
LLM	Large Language Model, велики језички модел
NLP	Natural Language Processing, обрада природног језика
OCR	Optical Character Recognition, оптичко препознавање знакова
RAG	Retrieval-Augmented Generation, генерисање уз помоћ претраживања
RRF	Reciprocal Rank Fusion, комбиновање реципрочних рангова
SDK	Software Development Kit, скуп алата за развој софтвера
SPLADE	Sparse Lexical and Expansion Model
UI	User Interface, кориснички интерфејс

САДРЖАЈ

1. Увод.....	8
1.1. Предмет и мотивација	8
1.2. Дефиниција проблема	8
1.3. Циљ и обухват рад	9
1.4. Границе примене система	9
1.5. Организација рада	9
2. Преглед области и постојећих приступа	10
2.1. Претрага документације и генерисање одговора	10
2.2. Развој семантичког проналажења информација	10
2.3. <i>Dense</i> и <i>sparse</i> претрага и комбиновање резултата	11
2.4. Избор приступа за практични систем	11
3. Теоријске основе	13
3.1. Структура RAG система	13
3.2. Векторизација и семантичка сличност	14
3.3. Мултијезична претрага и коришћени модели	14
3.4. Векторске базе и ChromaDB	15
3.5. Хибридна претрага и груписање	16
3.6. Сегментација и хијерархијски контекст	16
3.7. Велики језички модел и генерисање	17
4. Анализа проблема и пројектовање система	19
4.1. Корисници, задаци и границе примене	19
4.2. Структура и порекло корпуса	19
4.3. Функционални и нефункционални захтеви	20
4.4. Архитектура и границе одговорности	21
4.5. Модел података и следљивост	22
4.6. Пројектне одлуке и алтернативе	23
5. Имплементација система	25
5.1. Развојно окружење и организација кода	25
5.2. Учитавање PDF докумената	26
5.3. Препознавање секција	26
5.4. Parent-child сегментација	27
5.5. Припрема записа за Chroma Cloud	28
5.6. Векторизација, колекција и упис	28
5.7. Претрага и формирање retrieval излаза	29
5.8. Формирање контекста и позив Gemini модела	31
5.9. Кориснички интерфејс и историја	34
6. Тестирање и експериментални резултати	37
6.1. Софтверски тестови	37
6.2. Провера корпуса и сегментације	37
6.3. Методологија евалуације претраге	38
6.4. Резултати основног скупа	39
6.5. Додатне језичке провере	41
6.6. Провера генерисања и времена одзива	41
6.7. Анализа грешака и ваљаности закључака	42
7. Безбедност, ограничења и даљи развој	43
7.1. Стручна одговорност и коришћење извора	43
7.2. Преостала техничка ограничења	44

7.3. Приоритети за унапређење	45
8. Закључак	46
Литература	47

1. Увод

1.1. Предмет и мотивација

Рад у апотеци подразумева коришћење различитих врста стручне документације. У ту документацију спадају упутства за лекове, материјали о интеракцијама лекова и смернице добре апотекарске праксе. Иако су такви документи неопходни за проверу информација, њихово ручно прегледање може бити непрактично када корисник постави питање природним језиком и очекује да се релевантан податак пронађе у већој збирци докумената.

Проблем није ограничен на проналажење документа по називу. Једна тема може бити описана различитим изразима, док различити документи могу садржати сличне стандардизоване формулације. Назив лека, назив активне супстанце и опис питања могу упућивати на исту област, иако не представљају исти текстуални низ. Због тога је потребан поступак који повезује разумевање корисничког питања са претрагом садржаја докумената.

Велики језички модел може да формулише повезан одговор, али његово коришћење без спољног контекста не обезбеђује да је одговор заснован на конкретној документацији која је доступна у посматраном систему. *Retrieval-Augmented Generation*, односно генерисање уз помоћ претраживања, раздваја избор релевантног садржаја од његовог језичког представљања. У оригиналном раду о *RAG* приступу параметарско знање генеративног модела повезује се са непараметарском меморијом коју представља индекс спољних текстова [1].

Предмет овог рада је развој и анализа прототипа система који наведени принцип примењује на ограничен корпус докумената релевантних за рад у апотеци. Пројекат обрађује PDF документе, препознаје њихове типове и секције, формира мање сегменте за претрагу, складишти их у *Chroma Cloud* колекцији и на основу пронађеног контекста генерише одговор помоћу *Gemini* модела. Практични опис система заснован је на стварном изворном коду и сачуваним експерименталним артефактима пројекта [2].

1.2. Дефиниција проблема

Основни проблем који се решава јесте подршка кориснику при проналажењу релевантних делова фармацеутске документације и формулисању одговора на питање постављено природним језиком. Улаз система је питање корисника, док се одговор формира на основу садржаја који је претходно учитан и припремљен у бази знања.

Да би се овај проблем решио, потребно је обухватити више повезаних корака. Најпре се PDF документи морају учитати тако да се сачувају подаци о извору и страницама. Затим је потребно узети у обзир да различите групе докумената имају различиту структуру. Упутства за лекове садрже стандардне главне секције, Водич добре апотекарске праксе користи PDF обележиваче када су доступни, а документ о интеракцијама захтева правила за препознавање познатих наслова.

Након препознавања структуре, текст се дели на мање *child* сегменте који су погоднији за претрагу. Сваки сегмент задржава везу са широм *parent* секцијом, изворним PDF документом, страницама и, када је реч о упутству за лек, називом лека и активном супстанцом. На тај начин се резултат претраге не своди само на сличност текста, већ садржи и податке потребне за тумачење и приказ извора.

Проблем има и безбедносну и методолошку страну. Систем који проналази тематски сличан текст не мора нужно да је пронашао одговарајући лек или тачан доказ

за сваку тврдњу у одговору. Зато се резултати у овом раду тумаче као подршка претрази и читању документације, а не као аутоматска замена за стручну процену.

1.3. Циљ и обухват рад

Циљ рада је развој функционалног RAG прототипа који повезује обраду PDF докумената, мултијезичну векторизацију, *ChromaDB* претрагу и генерисање одговора. Документи се деле на секције (*parent* целине) и мање текстуалне сегменте (*child* сегменте), повезане идентификатором *parent_id* у метаподацима. Претрага и формирање контекста за генерисање заснивају се на *child* сегментима, док веза са секцијом омогућава праћење њиховог порекла.

Обухват реализације чине учитавање PDF страница, класификација докумената, препознавање секција и формирање *parent* и *child* сегмената. *Chroma Cloud* користи густу *Qwen* и ретку *SPLADE* репрезентацију, уз комбиновање рангова методом RRF. *Gemini* добија питање и ограничени контекст, а *Streamlit* приказује одговор, изворе и контекст у оквиру сесије. Јединични и интеграциони тестови допуњени су оценом претраге на референтном скупу питања.

Практични допринос рада обухвата раздвајање фаза обраде, правила за различите врсте докумената, *parent-child* организацију, додавање идентификационог контекста у текст сегмената и повезивање са спољним сервисима за векторизацију, претрагу и генерисање. Додатни допринос представљају тестови и евалуација, јер омогућавају да се разликују успешан пролазак програмског тока, проналажење очекиване секције и стварна поткрепљеност генерисаног одговора.

1.4. Границе примене система

Рад не представља развој новог језичког модела нити обучавање модела од почетка. У систему се користе већ доступне компоненте: *pypdf* за читање PDF докумената, *LangChain Document* објекти за заједнички облик текста и метаподатака, *Chroma Cloud* за векторску претрагу и *Gemini* за генерисање одговора.

Прототип није клинички валидиран и није испитано да ли су сви његови одговори медицински тачни. Корпус је ограничен на затечене документе, а основни референтни скуп садржи ограничен број питања. Такође, пројекат не води евиденцију пацијената, не садржи аутентификацију и систем улога, не поставља дијагнозе и не доноси терапијске одлуке.

Систем је намењен подршци претрази и разумевању документације. Одговор треба проверити у одговарајућем изворном документу, а одлуке које захтевају фармацеутску или лекарску процену остају у надлежности квалификованог стручњака. Ова граница је важна и за тумачење резултата: мерене вредности *retrieval* метрика не могу се тумачити као клиничка тачност или безбедност система.

1.5. Организација рада

После увода следе преглед постојећих приступа и теоријске основе RAG-а, векторизације, хибридне претраге и генерисања. Четврто поглавље описује проблем, корпус и архитектуру, а пето конкретну имплементацију. Шесто поглавље приказује софтверске провере и резултате евалуације. Седмо разматра безбедност, ограничења и правце развоја, док осмо доноси закључак. Литература и додаци садрже коришћене изворе, инвентар корпуса, тестна питања и команде за репродукцију.

2. Преглед области и постојећих приступа

2.1. Претрага документације и генерисање одговора

За постављени проблем могу се разматрати три основна приступа. Први је непосредна претрага текстова, при којој корисник добија листу докумената или пасуса и сам саставља одговор. Други је постављање питања генеративном моделу без спољног контекста. Трећи раздваја избор доказа од његовог језичког представљања и одговара RAG архитектури. То су архитектонске алтернативе, а не три експериментално упоређена производа у овом пројекту.

Непосредна претрага има важну предност: изворни текст остаје видљив и није преформулисан моделом. Међутим, поступак проналажења, поређења и сажимања остаје на кориснику. Генерисање без претраге може дати читљив одговор, али не успоставља контролисану везу са локалном збирком PDF-ова. Избор RAG-а у овом раду мотивисан је потребом да се језичка употребљивост повеже са проналажењем проверљивих докумената.

Оригинални RAG рад описује обучиве моделе и експерименте са индексом Wikipedia пасуса. Практични систем у овом раду не репродукује тај поступак обучавања: користи већ доступне моделе и програмски организован ток претраге и генерисања. Зато је прецизније говорити о примени RAG архитектонског принципа него о имплементацији целокупног алгорита обучавања из оригиналног рада [1].

2.2. Развој семантичког проналажења информација

Развој дубоког учења и савремених архитектура за обраду природног језика довео је до промене начина на који се текстуални садржај представља и претражује. Један од значајних корака представља *Transformer* архитектура, заснована на механизму пажње (енг. *attention*), која омогућава моделима да приликом обраде одређеног дела текста узму у обзир његов однос са другим деловима секвенце. Овакав начин обраде представља основу за развој бројних савремених језичких модела и метода за формирање текстуалних репрезентација [3].

На основу оваквих архитектура развијени су модели намењени формирању векторских репрезентација текста. Један од значајних примера је *Sentence-BERT*, који омогућава представљање реченица у облику вектора погодних за ефикасно семантичко поређење. За разлику од приступа који се ослањају искључиво на подударање речи, векторске репрезентације омогућавају поређење текстова на основу њиховог значења у заједничком векторском простору. Значај овог приступа за претрагу огледа се и у могућности да се репрезентације докумената израчунају унапред, док се репрезентација корисничког упита израчунава у тренутку претраге [4].

Даљи развој овог концепта представља *Dense Passage Retrieval (DPR)*, који примењује густе (енг. *dense*) векторске репрезентације за проналажење релевантних пасуса у колекцији докумената. Уместо да се цео документ представи једним вектором, садржај се може поделити на мање целине које се независно претражују. Овакав приступ је посебно значајан код обимне документације, јер један документ може садржати информације о великом броју различитих тема. Мањи сегменти омогућавају да резултат претраге буде усмерен на конкретан део садржаја који је релевантан за постављено питање [5].

Развој оваквих метода довео је до примене семантичке претраге у системима за проналажење информација. Уместо да се релевантност одређује само на основу присуства истих речи у упиту и документу, могуће је поредити њихове векторске

репрезентације и проналазити садржај који је семантички повезан са постављеним питањем.

2.3. *Dense* и *sparse* претрага и комбиновање резултата

У претходном одељку описан је развој семантичког проналажења информација и значај векторских репрезентација за претрагу садржаја на основу значења. Иако *dense* приступ омогућава проналажење семантички повезаних информација, ослањање на само један тип претраге није увек довољно за све врсте упита. У зависности од садржаја и природе упита, релевантност документа може зависити како од његовог семантичког значења, тако и од присуства конкретних термина који се појављују у упиту.

Због тога се у савременим системима за проналажење информација разматрају различити типови текстуалних репрезентација и начина претраге. *Dense* приступ усмерен је пре свега на семантичку повезаност садржаја, док *sparse* приступ задржава израженију везу са лексичким карактеристикама текста.

Један од приступа који развија *sparse* претрагу је *SPLADE*. Овај приступ користи језички модел за формирање *sparse* репрезентација и проширивање лексичког представљања текста. На тај начин претрага задржава повезаност са конкретним терминима, али истовремено користи научене репрезентације које омогућавају повезивање сродних појмова [6].

Dense и *sparse* претрага могу дати различите, али међусобно допуњујуће резултате. *Dense* претрага може препознати семантичку повезаност и када се у упиту и документу не користе потпуно исти термини, док *sparse* приступ може бити посебно значајан када су конкретне речи и стручни термини важни за проналажење релевантног садржаја.

Комбиновање резултата различитих претрага захтева посебан механизам рангирања, јер различити претраживачи могу користити различите начине израчунавања оцена релевантности. Директно сабирање њихових оцена зато није увек одговарајуће. Један од приступа за решавање овог проблема представља *Reciprocal Rank Fusion (RRF)*, који резултате комбинује на основу њихових позиција на појединачним ранг-листама. Резултати који се налазе високо на више листа на тај начин добијају већи укупни значај [7].

Предност оваквог приступа је у томе што није неопходно директно упоређивати оцене које потичу из различитих система претраге. Уместо тога, користи се информација о рангу сваког резултата, чиме се омогућава спајање више независних листа у јединствену листу кандидата. *RRF* се због тога може користити као механизам за обједињавање резултата различитих система претраге.

Комбиновање *dense* и *sparse* претраге на овај начин представља основу хибридне претраге. Она настоји да искористи предности оба приступа: семантичку повезаност коју обезбеђује *dense* претрага и лексичку специфичност *sparse* претраге.

2.4. Избор приступа за практични систем

Изабрана архитектура комбинује структуру извора и векторску претрагу. Познате границе секција користе се пре векторизације, док модел претраге решава који је сегмент најближи конкретном питању.

За упутство за лек посебно је важно да сегмент има идентитет лека. Текстови различитих упутстава често садрже сличне формулације и исте главне наслове. Уколико је представљено само тело пасуса, добро тематско поклапање може припадати

погрешном документу. Зато у овом пројекту назив лека и његов садржај постају део текста који се векторизује. Ова одлука је разумљива по конструкцији система, али њен изоловани допринос није измерен посебним експериментом са и без префикса.

Parent секција представља виши ниво организације документа и повезана је са својим *child* сегментима преко идентификатора *parent_id*. *Child* сегменти се користе као јединице претраге и као текстуални контекст који се прослеђује генеративном моделу. *Parent* секције се не користе за замену пронађеног *child* текста целокупном секцијом, већ служе за очување структурне везе, идентификацију изворне секције и праћење порекла резултата. Оваква организација смањује количину непотребног текста у промπτу, али може довести до губитка ширег контекста који се налази ван пронађеног *child* сегмента. *Parent-child* приступ зато није аутоматско решење свих проблема сегментације, већ компромис између прецизности претраге, количине контекста и очувања шире структуре документа.

3. Теоријске основе

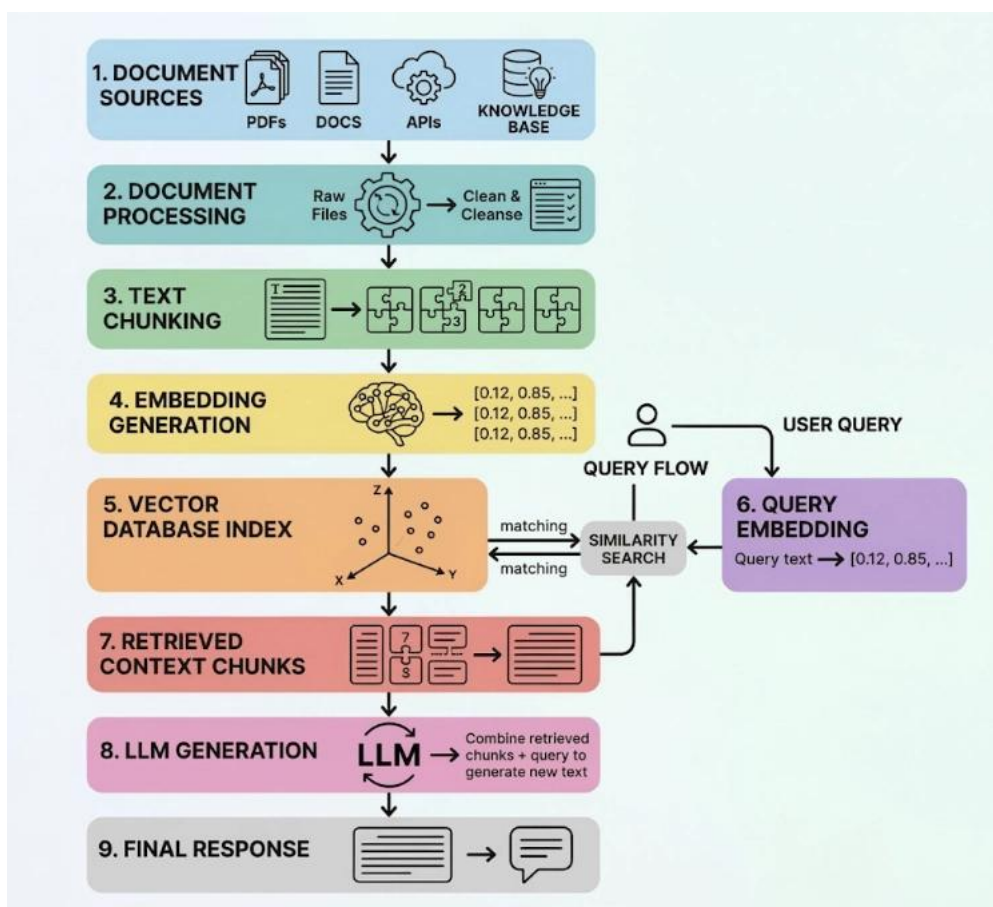
3.1. Структура RAG система

RAG систем се састоји од више међусобно повезаних компоненти чији је заједнички циљ да генеративном моделу обезбеде релевантне информације из спољашњег извора знања. У општем случају, архитектура RAG система може се поделити на две основне фазе: фазу припреме базе знања и фазу одговарања на кориснички упит.

У фази припреме базе знања, изворни документи се учитавају и обрађују, након чега се деле на мање текстуалне целине. Добијени сегменти се трансформишу у векторске репрезентације помоћу *embedding* модела и смештају у одговарајући индекс или векторску базу података. Ова фаза се обично извршава пре интеракције са корисником, тако да није неопходно поново обрађивати и векторизовати целокупан корпус приликом сваког новог питања.

У фази одговарања, кориснички упит се најпре обрађује и користи за проналажење релевантних сегмената из припремљене базе знања. Компонента задужена за претрагу, односно retriever, на основу корисничког упита рангира релевантне сегменте према њиховој релевантности за постављени кориснички упит. Одабрани сегменти се затим прослеђују компоненти за формирање контекста, која их комбинује са корисничким упитом и системским инструкцијама.

Тако формиран контекст представља улаз генеративном моделу. Генеративна компонента, најчешће велики језички модел (LLM), на основу корисничког упита, достављеног контекста и дефинисаних инструкција генерише коначан одговор.



Слика 3.1. Општа архитектура RAG система

Нека је D корпус докумената, q корисничко питање, R поступак претраге, B поступак формирања контекста, а G генеративна компонента. Тада се ток одговарања може формално представити као:

$$C = B(R(q, D)), \quad a = G(q, C, I) \quad (1)$$

где је I системска инструкција, C формирани контекст, а a генерисани одговор. Овај запис представља апстрактан приказ тока података кроз RAG систем, при чему се у процесу одговарања најпре врши претрага релевантних информација, затим формирање контекста, а потом његово прослеђивање генеративном моделу.

3.2. Векторизација и семантичка сличност

Нумеричка, односно векторска репрезентација текста којом се информација о његовом значењу представља у вишедимензионалном векторском простору назива се *embedding*. Текстови са сличним значењем настоје да буду представљени векторима који су међусобно блиски у том простору. Векторска репрезентација текста добија се применом *embedding* модела, односно модела који текстуални улаз трансформишу у вектор фиксне димензије.

Векторизација текста представља сам поступак трансформације текстуалног садржаја у овакву нумеричку репрезентацију. За *dense* векторске репрезентације може се записати $f(d) \in \mathbb{R}^m$, где је d текстуални улаз, а m димензија добијеног вектора. Таква репрезентација омогућава да се текстуални садржаји пореде на основу њихове семантичке сличности, што представља основу за *dense* претрагу у RAG систему.

Векторска репрезентација не представља замену за оригинални текст, јер се из *embedding*-а у посматраном систему не реконструише изворни текстуални садржај. Због тога систем, поред векторских репрезентација, задржава оригинални текст и одговарајуће метаподатке у бази.

Упит и документ морају бити представљени компатибилним поступком. Иста димензија два вектора није довољна ако потичу из различитих модела или различито конфигурисаних простора. Промена модела за документе, без одговарајуће промене репрезентације питања, нарушила би смисао поређења. То је разлог да се конфигурација *embedding* модела посматра као део верзије индекса.

У конкретној имплементацији за *dense* индекс користи се косинусна сличност:

$$\cos(u, v) = \frac{\sum_{j=1}^m u_j v_j}{\sqrt{\sum_{j=1}^m u_j^2} \sqrt{\sum_{j=1}^m v_j^2}} \quad (2)$$

Косинусна сличност пореди оријентацију ненултих вектора. Када API врати косинусно растојање, апликација израчунава оцену као $1.0 - \text{distance}$. Ова оцена не представља вероватноћу да је коначни одговор тачан, већ меру сличности која се користи за поређење кандидата у оквиру претраге.

3.3. Мултијезична претрага и коришћени модели

Мултијезична претрага представља приступ у којем систем омогућава проналажење семантички релевантних докумената без обавезног подударања језика између корисничког упита и документа. За RAG системе ова особина је значајна зато што кориснички упит и документи могу бити написани на различитим језицима, односно могу садржати термине, скраћенице или називе који потичу из различитих

језичких контекста.

Qwen3-Embedding-0.6B је модел намењен представљању текста за *embedding* задатке. Ауторска документација наводи мултијезичну подршку за више од 100 језика, максималну димензију репрезентације 1.024 и подршку за инструкције задатка [8]. У раду се за генерисање *embedding* репрезентација докумената користи инструкција “Represent this Serbian pharmacy document chunk for retrieval.”. Инструкција моделу експлицитно описује улогу улазног текста, односно да се прослеђени сегмент документа представља за потребе претраге. На тај начин се *embedding* моделу пружа додатни контекст о намени текста у *retrieval* задатку.

У имплементацији бира се *QWEN3_EMBEDDING_0p6B* преко *ChromaCloudQwenEmbeddingFunction*. Ова интеграција позива удаљени *Chroma* сервис, па локални програм не учитава тежине *Qwen* модела у GPU меморију. Инструкције за документ и питање су различите, али оба улаза припадају истом *retrieval* задатку. Доступност сервиса и његова ограничења зависе од *Cloud* конфигурације [9].

За ретке векторе користи се *ChromaCloudSpladeEmbeddingFunction* са избором *SPLADE_PP_EN_V1*. И ово израчунавање обавља удаљени сервис. Када програм позове *embedding* функцију у SDK-у, то не значи да је неуронски модел покренут у локалном *Python* процесу [10].

Мултијезичност *embedding* модела не подразумева нужно идентично понашање за све језике и све облике текста. Разлике у језичкој структури, писму, морфологији, терминологији и начину представљања речи могу утицати на квалитет векторских репрезентација и резултате претраге. Због тога је подршку за више језика потребно посматрати као својство модела које омогућава примену на различитим језицима, а не као гаранцију једнаке успешности *retrieval*-а за сваки језик и сваки тип упита.

3.4. Векторске базе и ChromaDB

Векторске базе представљају системе за складиштење и претрагу података чија је основа векторска репрезентација садржаја. За разлику од традиционалних база података, код којих се претрага најчешће заснива на тачном поклапању вредности, условима над атрибутима или текстуалним операторима, векторске базе омогућавају проналажење записа на основу сличности између њихових векторских репрезентација. Ова особина их чини погодним за *retrieval* фазу RAG система, у којој је потребно пронаћи делове докумената који су семантички повезани са корисничким упитом.

ChromaDB је база података намењена раду са *embedding* репрезентацијама и *retrieval* задацима. *Chroma* организује записе у колекције. Запис се у овом пројекту посматра као веза идентификатора, текста, метаподатака и векторских репрезентација. *get_or_create_collection* омогућава отварање постојеће или креирање именоване колекције [11].

Метаподаци представљају додатне структуриране информације повезане са записом. Они могу описивати порекло документа, врсту записа, припадност одређеној секцији или друге атрибуте значајне за организацију и филтрирање података. Важно је разликовати претрагу по векторској сличности од филтрирања по метаподацима. Само постојање одређеног метаподатка не подразумева да ће он аутоматски утицати на резултат претраге. Услов над метаподацима мора бити експлицитно задат приликом извршавања упита.

Schema описује индексе над пољима записа. За ретки индекс конфигурише се посебан кључ, извор текста и *embedding* функција. То омогућава да један документ има *dense* и *sparse* репрезентацију за претрагу [12].

3.5. Хибридна претрага и груписање

Хибридна претрага представља приступ у којем се комбинују резултати добијени различитим методама претраге. У RAG системима могу се комбиновати *dense* и *sparse* претрага, јер се ове методе заснивају на различитим начинима одређивања релевантности докумената. *Dense* претрага омогућава проналажење садржаја на основу семантичке сличности, док *sparse* претрага у већој мери користи лексичке карактеристике текста. Комбиновањем резултата ових метода могуће је добити јединствено рангирање које користи предности оба приступа.

Један од начина комбиновања различитих ранг-листа је *Reciprocal Rank Fusion* (RRF). За сваки документ посматра се његов ранг у резултатима сваке појединачне претраге. Документ добија већи допринос коначној оцени уколико се налази високо на више ранг-листа. Код пондерисаног RRF-а различитим методама претраге могу се доделити различите тежине. За *Chroma* варијанту пондерисаног RRF-а оцена може бити записана као:

$$s_{Chroma}(d) = - \sum_{i \in \{dense, sparse\}} \frac{w_i}{c + r_i(d)} \quad (3)$$

где је $r_i(d)$ ранг кандидата у одговарајућој листи, w_i тежина одговарајуће методе претраге, а c параметар који ублажава утицај ранга. Документ који није присутан у једној листи нема допринос из те листе, што се у формули може представити бесконачним рангом.

Конкретна реализација RRF-а може се разликовати у зависности од система који га имплементира. *Chroma* користи сопствену конвенцију за представљање RRF резултата, укључујући негативну вредност оцене, при чему мања вредност означава бољи резултат. Због тога је приликом тумачења резултата потребно узети у обзир конкретну имплементацију и API који се користи [13].

Поред комбиновања резултата различитих метода претраге, могуће је применити и груписање пронађених резултата према одређеном атрибуту. Груписање омогућава да се резултати расподеле у групе на основу заједничке особине, након чега се може ограничити број резултата који потичу из једне групе. На тај начин могуће је спречити да један извор заузме превелики део коначне листе резултата.

Chroma Search API обезбеђује механизме за груписање резултата, при чему параметри као што су *GroupBy* и *MinK* омогућавају контролу броја резултата који се задржавају унутар група [14]. Груписање може повећати разноврсност извора у коначној листи, али истовремено може довести до изостављања додатних релевантних резултата уколико се већи број релевантних сегмената налази у истој групи.

3.6. Сегментација и хијерархијски контекст

Сегментација текста (енг. *chunking*), представља поступак поделе документа на мање текстуалне целине које се могу појединачно индексирати и претраживати. Избор величине и граница сегмената директно утиче на квалитет *retrieval*-а. Превелик сегмент може обухватити више различитих тема и тиме смањити прецизност претраге, док премали сегмент може изгубити део контекста неопходног за правилно разумевање садржаја. Због тога се сегментација може заснивати на структурним границама документа, као што су наслови и секције, а затим додатно ограничити максималном величином текста [15].

Један од приступа организацији сегмената је хијерархијска, односно *parent-child* структура. *Parent* документ представља ширу логичку целину, док *child* документ представља мањи део те целине који се користи као јединица претраге. Веза између

parent и *child* докумената може се успоставити путем идентификатора који се чува у метаподацима *child* документа. На тај начин могуће је задржати информацију о припадности мањег сегмента ширем контексту, док се *retrieval* може извршавати над мањим *child* сегментима.

Приликом сегментације може се користити преклапање суседних сегмената (енг. *overlap*). Оно подразумева да се део текста са краја једног сегмента понови на почетку наредног сегмента. Циљ овог поступка је смањење губитка контекста на границама сегмената, посебно када се важна информација простира преко границе између два суседна сегмента. Веће преклапање, међутим, повећава количину дуплираног текста и број потенцијално сличних резултата. Само преклапање не гарантује очување целе реченице, пасуса или структуре табеле, па коначни резултат зависи и од начина на који је реализована функција за поделу текста.

3.7. Велики језички модел и генерисање

Велики језички модели (енг. *Large Language Models*, LLM) представљају моделе машинског учења намењене обради и генерисању текста. Они текст обрађују у облику токена, који могу представљати целе речи, делове речи или друге мање текстуалне целине. На основу улазних токена и параметара научених током обучавања, модел генерише излазни текст предвиђањем наредних токена. Због начина токенизације, број знакова или речи у тексту није директно једнак броју токена које модел обрађује.

У RAG систему генеративни модел не добија само корисничко питање, већ и контекст добијен у *retrieval* фази. Тај контекст обично садржи одабране сегменте из базе докумената који су релевантни за постављено питање. Генеративни модел затим користи кориснички упит и достављени контекст за формирање одговора. На овај начин *retrieval* компонента одређује који делови расположивих докумената ће бити доступни генеративном моделу, док LLM врши њихову интерпретацију и генерисање коначног текста.

У систему је за генерисање одговора изабран модел *gemini-3.6-flash*. Наведени идентификатор одговара ознаци модела наведеној у званичној документацији компаније Google [16].

Начин формирања улазног контекста има директан утицај на генерисани одговор. Контекст мора бити довољно информативан да садржи податке релевантне за питање, али његова величина истовремено је ограничена контекстним прозором модела и практичним ограничењима система. Због тога се у RAG архитектури настоји да се моделу проследи релевантни сегменти, уместо целокупног корпуса. Поред садржаја контекста, на генерисање могу утицати и системска инструкција, кориснички упит и параметри који контролишу дужину и начин генерисања одговора.

Системска инструкција представља текстуално упутство којим се моделу дефинишу правила за генерисање одговора. У RAG систему она може одредити начин коришћења достављеног контекста, језик одговора, формат излаза и понашање у случају када релевантне информације нису присутне у достављеним документима. Међутим, инструкција представља смерницу за понашање модела, а не формални механизам верификације. Генерисани текст зато не треба посматрати као аутоматски проверену репродукцију садржаја извора.

Посебно је важно разликовати *retrieval* и генеративну фазу RAG система. *Retrieval* одређује које информације из корпуса улазе у контекст, док генеративни модел на основу тог контекста формулише одговор. Грешка у *retrieval* фази може довести до тога да модел не добије релевантан део извора, док грешка у генеративној фази може настати и када је релевантан контекст исправно достављен. Ово раздвајање

омогућава да се квалитет RAG система анализира одвојено на нивоу претраге и на нивоу генерисања.

4. Анализа проблема и пројектовање система

4.1. Корисници, задаци и границе примене

Предвиђени корисник система је стручна особа која претражује фармацеутску документацију и која може да провери добијени одговор у односу на изворни документ. У контексту овог рада то је пре свега фармацеут. Систем не предвиђа различите корисничке улоге, већ свим корисницима обезбеђује исти основни начин претраге и генерисања одговора.

Основни задатак система је проналажење релевантних информација у доступном корпусу докумената и њихово представљање у облику одговора на корисничко питање. Питања се могу односити на конкретан део документа, одређену тему или појам, као и на поступке описане у стручним приручницима. У зависности од типа питања, релевантност резултата може зависити од идентификације документа и теме, препознавања кључног појма или проналажења одговарајућег дела структурисаног приручника.

Обухват система одређен је структуром и садржајем корпуса који се користи за претрагу. Различите врсте фармацеутске документације могу имати различите структуре и намене. На пример, упутство за пацијента и сажетак карактеристика лека представљају различите типове докумената и не морају имати исту структуру. ЕМА QRD образац описује структуру ових докумената, што је релевантно за одређивање типа документа који систем обрађује [17]. Због тога структурна правила примењена у систему не треба тумачити као општи поступак за обраду свих врста медицинске документације.

Граница примене система постављена је на претрагу доступних докумената и представљање информација садржаних у њима. Систем није намењен издавању рецепата, самосталној процени терапије на основу комплетног картона пацијента, управљању залихама или приступу апотекарском информационом систему. Такође, питања која захтевају податке о конкретном пацијенту, као што су анамнеза, лабораторијски налази или процена хитности, превазилазе информације које се могу поуздано добити само претрагом опште документације. Генерисани одговор се зато посматра као информација заснована на доступном корпусу, а не као замена за стручну процену.

4.2. Структура и порекло корпуса

Корпус система састоји се од PDF докумената који су након прикупљања и структурне припреме организовани у *parent* секције и *child* сегменте. Састав корпуса и резултат његове структурне припреме приказани су у табели 1. Табела приказује број PDF докумената, укупан број страница, број издвојених *parent* секција и број добијених *child* сегмената за сваку врсту документа.

Табела 1. Састав корпуса и резултат структурне припреме

Тип	Број PDF-ова	Број страница	parent секције	child сегменти
Упутства за лекове	29	226	203	741
Водич добре апотекарске праксе	1	93	279	536

Тип	Број PDF-ова	Број страница	parent секције	child сегменти
Референца о интеракцијама	1	20	11	84
Укупно	31	339	493	1.361

Корпус је формиран од три групе докумената. Упутства за лекове коришћена у корпусу преузета су са званичне странице Агенције за лекове и медицинска средства Србије (АЛИМС) [18]. Водич добре апотекарске праксе преузет је са веб-сајта Министарства здравља Републике Србије [19], док је документ о интеракцијама лекова преузет са веб-сајта Фармацеутске коморе Србије [20].

4.3. Функционални и нефункционални захтеви

Функционални захтеви обухватају припрему базе знања и одговарање на питања корисника. Припрема почиње учитавањем PDF докумената из изабраног директоријума и његових поддиректоријума. Систем издваја текст страница, бележи податке о извору и разликује типове докумената како би применио одговарајућа правила за препознавање секција. Текст сваке секције затим се дели на мање сегменте, уз очување везе са секцијом и документом из којег потиче. Припремљени текстови се векторизују и, заједно са метаподацима, чувају у бази ради касније претраге.

Након уноса корисничког питања, систем претражује сачуване сегменте и издваја најбоље рангиране резултате. Подржане су *dense* претрага и хибридна претрага, која комбинује *dense* и *sparse* векторске репрезентације. Изабрани сегменти формирају контекст ограничене дужине који се, заједно са питањем и системском инструкцијом, прослеђује генеративном моделу.

Кориснички интерфејс омогућава постављање питања, читање генерисаног одговора и преглед пронађених извора и сегмената. Приказ изворног текста омогућава кориснику да упореди одговор са садржајем документације. Преглед наведених функционалних захтева и механизма њихове реализације дат је у табели 2.

Табела 2. Функционални захтеви и њихова реализација

Захтев	Реализовани механизам
Учитавање PDF докумената из стабла директоријума	Функције <code>load_all_documents</code> и <code>load_pdf</code> издвајају текст страница и придружују податке о извору.
Разликовање типова докумената	<code>DocumentType</code> и <code>detect_document_type</code> омогућавају избор правила обраде према типу документа
Препознавање секција документа	<code>build_parent_sections</code> издваја тематске целине на основу наслова, PDF обележивача и правила за одговарајући тип документа
Формирање мањих сегмената уз очување везе са секцијом	<code>build_parent_child_documents</code> формира <code>parent</code> секције и <code>child</code> сегменте, повезане метаподатком <code>parent_id</code>
Векторизација и складиштење припремљених текстова	<code>Chroma Cloud embedding</code> интеграције за <code>Qwen</code> и <code>SPLADE</code> формирају векторске репрезентације, а текстови и метаподаци уписују се у колекцију

Захтев	Реализовани механизам
Проналажење релевантних сегмената за корисничко питање	ChromaVectorStore.search подржава густу и хибридную претрагу
Избор контекста за генерисање одговора	ChromaRetriever враћа одабране child сегменте, а build_rag_prompt и format_context повезују њихов текст са питањем уз ограничење дужине контекста
Генерисање одговора на основу пронађеног текста	RAGPipeline повезује претрагу и генерисање, а GeminiFlashClient прослеђује моделу питање, контекст и системску инструкцију
Приказ одговора, извора и пронађеног текста	Streamlit интерфејс приказује одговор, табелу извора и прошириви приказ пронађених сегмената

Нефункционални захтеви односе се на својства система која нису директно везана за појединачну функцију, већ одређују начин његовог рада и одржавања. У овом систему они обухватају разумљивост и одрживост кода, следљивост резултата, поновљивост припреме корпуса и контролисану употребу спољних сервиса.

4.4. Архитектура и границе одговорности

Систем је реализован као модуларна *Python* апликација са *Streamlit* корисничким интерфејсом и интеграцијом са два спољна сервиса, *Chroma Cloud* и *Google Gen AI* сервисом. Архитектура је организована тако да су припрема докумената, индексирање, *retrieval* и генерисање одговора раздвојени у логичке целине. На тај начин свака компонента има јасно дефинисану одговорност и може се посматрати независно од осталих делова система.

Кориснички интерфејс реализован у *Streamlit*-у прихвата питање корисника и приказује генерисани одговор, изворе и пронађене сегменте. Апликациони слој повезује кориснички интерфејс са компонентама за *retrieval* и генерисање. *Retrieval* компонента комуницира са *Chroma Cloud* сервисом ради претраге индексираних *child* сегмената, док *RAGPipeline* повезује резултате претраге са генеративном фазом. Генеративни клијент прослеђује корисничко питање, одабрани контекст и системску инструкцију *Google Gen AI* сервису и преузима генерисани одговор.

Посебна целина система намењена је припреми корпуса. Она обухвата читавање PDF докумената, препознавање њиховог типа, издвајање секција и формирање *parent-child* структуре. Припремљени текстови и метаподаци затим се прослеђују компоненти за векторизацију и упис у *Chroma Cloud*.



Слика 4.1. Архитектура реализованог система, ауторска текстуална шема на основу кода

Раздвајање компоненти дефинише и границе њихових одговорности. Компонента за припрему докумената не одлучује о садржају коначног одговора, *retrieval* компонента не генерише текст, а генеративна компонента не врши самостално претрагу корпуса. На овај начин ток података пролази кроз јасно одвојене фазе: припрема корпуса, индексирање, *retrieval*, формирање контекста и генерисање одговора.

Модуларна организација омогућава и независно испитивање појединачних компоненти. Парсер секција може се испитивати над локалним страницама без приступа бази, функције за припрему *Chroma* записа над синтетичким *Document* објектима, док *Retriever* и *RAGPipeline* могу користити замењене зависности. Оваква организација олакшава изоловање узрока грешке и проверу понашања појединачних делова система.

4.5. Модел података и следљивост

LangChain се користи кроз стандардну репрезентацију текста и метаподатака у *Document* објекту. То омогућава да странице, секције и сегменти деле исти основни интерфејс [21].

Значење објекта зависи од фазе. *Document* који излази из *loader*-а представља једну непразну страницу. *Parent Document* представља препознату секцију. *Child Document* представља мањи део секције са префиксом.

Поред текста, сваки *Document* објекат садржи метаподатке који описују његово порекло и место у структури документа. Они омогућавају повезивање сегмента са изворним PDF-ом и припадајућом секцијом, као и приказ података о извору уз резултат претраге. Најважнија поља метаподатака и њихове улоге приказани су у табели 3.

Табела 3. Најважнији метаподаци и њихово значење

Поље	Ниво и улога
<i>source, file_name, folder</i>	Путања, назив PDF извора и назив родитељског директоријума
<i>source_document_id</i>	Идентификација изворног PDF документа; заједничка је за записе који потичу из истог документа
<i>document_type</i>	Одређује породицу правила која се користи за препознавање структуре документа
<i>medicine_name, active_substance</i>	Подаци из имена датотеке и директоријума; користе се првенствено за упутства за лекове
<i>page, page_number, page_index</i>	Број и индекс PDF странице; у каснијим фазама могу бити наслеђени од изворних записа на нивоу странице
<i>page_numbers</i>	Странице које обухвата једна <i>parent</i> секција
<i>section_id, parent_id</i>	Идентификатор секције и веза <i>child</i> записа са одговарајућом <i>parent</i> секцијом
<i>section_title, section_path</i>	Наслов секције и њена хијерархијска путања у документу
<i>section_detection</i>	Начин на који је препозната граница секције
<i>record_type</i>	Тип записа: <i>parent</i> или <i>child</i>
<i>child_chunk_index, child_chunk_count</i>	Редни број <i>child</i> сегмента и укупан број <i>child</i> сегмената у <i>parent</i> секцији
<i>cloud_subchunk_index</i>	Редни број физичког дела текста када је запис због ограничења величине подељен на више <i>Cloud</i> записа

4.6. Проектне одлуке и алтернативе

При пројектовању система размотрени су начини припреме базе знања, складиштења података и обраде корисничких питања. Изабрана решења повезују очување структуре докумената са претрагом мањих делова текста, уз раздвајање одговорности појединачних компоненти. У наставку су образложене најважније одлуке, њихова ограничења и могуће алтернативе.

У оквиру припреме базе знања, логичка структура документа издваја се пре примене ограничења дужине сегмената. На основу препознатих граница формирају се *parent* секције, а затим се њихов текст дели на мање *child* сегменте погодне за векторизацију и претрагу. Ово ограничење односи се на величину сегмената приликом припреме података, а не на дужину контекста који се касније прослеђује генеративном моделу. Правила за препознавање структуре прилагођена су типу документа, па се наслови водича не обрађују на исти начин као стандардне секције упутства. Такав поступак зависи од особина корпуса, али омогућава јасније праћење начина на који су сегменти настали. Алтернатива је јединствена подела свих докумената само према дужини текста, при којој би сегмент могао да обухвати садржај различитих секција.

Очување контекста издвојеног сегмента подржано је додавањем идентификационог префикса његовом тексту. Префикс садржи назив документа и

секције, као и додатне податке о предмету документа када су доступни. За разлику од података сачуваних искључиво у метаподацима, овај садржај улази у текст који *embedding* модел обрађује. Тиме се векторска репрезентација формира на основу пасуса и његових идентификационих података. Последица је повећање дужине сваког записа, а код веома кратких сегмената префикс може чинити значајан део укупног текста.

Parent секције и *child* сегменти чувају се у истој *Chroma* колекцији, а разликују се метаподатком *record_type*. Заједничка колекција поједностављује управљање записима, док *parent_id* повезује сваки *child* сегмент са припадајућом *parent* секцијом. Претрага је ограничена на *child* записе, чији се текст непосредно користи за формирање контекста. Пошто обе врсте записа пролазе кроз исти поступак уписа, векторизују се и *parent* текстови, иако њихови вектори не учествују у овој претрази. Засебно текстуално складиште *parent* секција могло би да избегне ту додатну обраду, али би захтевало управљање подацима у два складишта.

Обрада питања организована је као унапред одређен низ корака: претрага сегмената, припрема контекста и позив генеративног модела када контекст постоји. За разлику од агентског приступа, у коме модел може да одлучује о наредним корацима и покреће додатне претраге, овде током извршавања није предвиђен такав избор. То олакшава тестирање и анализу грешака, али ограничава прилагођавање сложенијим питањима. Систем зато не поставља самостално допунска питања, не понавља претрагу на основу процене непотпуности одговора и не проверава генерисане тврдње другим моделом.

5. Имплементација система

5.1. Развојно окружење и организација кода

Пројекат је реализован у програмском језику *Python*, уз коришћење библиотека *chromadb*, *google-genai*, *httpx*, *langchain-core*, *python-dotenv*, *pypdf* и *streamlit*. Коришћене верзије кључних библиотека су: *chromadb 1.5.9*, *google-genai 2.21.0*, *langchain-core 1.5.3*, *pypdf 6.15.0* и *streamlit 1.63.0*.

Изворни код је организован по функционалним целинама. Модули за учитавање и анализу докумената одвојени су од модула за сегментацију, припрему *Chroma* записа, претрагу и генеративно одговарање. Оваква организација омогућава да се појединачне фазе система тестирају и анализирају независно, преглед модула је приказан у табели 4.

Табела 4. Преглед модула и њихових одговорности

Модул	Одговорност
<code>data_loader.py</code>	Учитавање PDF страница, издвајање текста и формирање основних метаподатака
<code>document_types.py</code>	Препознавање типа документа и избор одговарајуће породице правила за даљу обраду
<code>document_sections.py</code>	Препознавање и формирање логичких <i>parent</i> секција на основу структуре документа
<code>chunking.py</code>	Формирање <i>child</i> сегмената унутар <i>parent</i> секција, додавање <i>retrieval</i> префикса и успостављање <i>parent-child</i> везе.
<code>chroma_documents.py</code>	Припрема записа за <i>Chroma Cloud</i> , контрола величине текста, чишћење метаподатака и формирање идентификатора
<code>vector_store.py</code>	Креирање <i>Chroma Cloud</i> колекције, <i>dense</i> и <i>sparse</i> индекса, упис записа и претрага <i>child</i> сегмената
<code>retriever.py</code>	Танак <i>retrieval</i> слој који прослеђује питање складишту и резултате претвара у објекте <i>RetrievedDocument</i>
<code>rag_prompt.py</code>	Формирање контекста од <i>retrieved child</i> сегмената, припрема <i>prompt</i> -а и издвајање информација о изворима
<code>gemini_client.py</code>	Позив <i>Google Gen AI SDK</i> -а и <i>Gemini</i> модела за генерисање одговора
<code>rag_pipeline.py</code>	Оркестрација целокупног RAG тока: <i>retrieval</i> , формирање <i>prompt</i> -а и генерација одговора
<code>ui_app.py</code>	Реализација <i>Streamlit</i> интерфејса, унос питања, приказ одговора, извора и <i>retrieved</i> контекста

5.2. Учитавање PDF докумената

Функција *load_pdf* најпре проверава да ли прослеђена путања постоји, да ли представља датотеку и да ли има PDF екстензију. Затим отвара документ помоћу класе *PdfReader*, издваја узорак текста са прве две странице ради препознавања типа документа и пролази кроз све странице PDF-а. За сваку страницу са успешно издвојеним и непразним текстом формира се један почетни, *page-level* објекат *Document*.

Поступак издвајања текста једноставан је и приказан у листингу 5.1. Функција користи постојећи текстуални слој PDF документа. Документација библиотеке *pypdf* разликује издвајање текста из PDF-а од препознавања текста који се налази у слици [22].

Листинг 5.1. Издвајање текста странице из *data_loader.py*

```
def _extract_page_text(page: Any) -> str:
    try:
        return (page.extract_text() or "").strip()
    except Exception as exc:
        print(f"[WARN] Could not extract page text: {exc}")
        return ""
```

Ако издвајање текста не успе, функција враћа празан стринг. На тај начин тренутна страница се прескаче, а обрада осталих страница може да се настави. Слично томе, функција *load_all_documents* хвата грешку на нивоу појединачног PDF документа, исписује упозорење и наставља обраду следећег документа. Овакво понашање је практично за демонстрациони *ingestion*, али може довести до непотпуног корпуса без статуса који би означио да је цела команда неуспешна. Због тога је потребно пратити упозорења и упоредити број очекиваних са бројем успешно обрађених датотека и страница.

Функција *detect_document_type* одређује тип документа на основу његове путање и узорка текста. Редослед правила је значајан: најпре се проверавају обележја водича и документа о интеракцијама, затим присуство директоријума *Lekovi*, а потом текстуални маркер који указује на упутство за лек. Документ који не одговара ниједном од ових правила добија тип *unknown*. Ова класификација је хеуристичка, јер се заснива на називу путање и узорку текста, а не на обученом класификатору.

Назив лека добија се из назива PDF датотеке без екстензије. Подвлаке и цртице претварају се у размаке, а прво слово се, када је потребно, пише великим словом. Активна супстанца добија се из назива непосредног родитељског директоријума. На тај начин организација локалних датотека добија употребљиву улогу у формирању метаподатака и *retrieval* префикса. Ипак, у систему нису реализовани аутоматска корекција правописа, препознавање синонима нити фармацеутска валидација назива.

У текућем корпусу свака страница дала је непразан текст. Тај резултат показује да је постојао читљив текстуални слој.

5.3. Препознавање секција

Функција *build_parent_sections* групише странице према изворном PDF документу и уређује их по редоследу страница. Сваки PDF обрађује се независно, па *parent* секција не може настати спајањем краја једног и почетка другог документа. Текстови страница се затим повезују, док *TextSpan* објекти памте позиције у комбинованом тексту и бројеве страница на које се ти делови односе.

За упутства за лекове користе се шаблони шест главних наслова. Нормализовани

текст олакшава поређење различитих размака, интерпункције и латиничких дијакритичких знакова, а посебни шаблони обухватају и ћириличне формулације. Поред нормализованог текста, чува се мапа оригиналних позиција. То је важно зато што се коначна *parent* секција издваја из оригиналног текста, а не из нормализоване копије.

Упутства често у уводном делу набрајају наслове који се касније понављају као стварне границе поглавља. Функција *_select_last_complete_leaflet_sequence* зато уназад тражи последњи потпуни редослед од шесте до прве секције. Она не бира прво појављивање сваког броја. Текст испред прве изабране секције остаје као *Document introduction*. У провереном корпусу свако од 29 упутстава на тај начин даје седам *parent* секција: увод и шест главних секција.

Овај алгоритам је детерминистички, али има услове успешности. Потребно је да свих шест образаца буде пронађено и да њихов редослед одговара претпостављеној структури. Ако један од наслова недостаје, користи се подела по страницама. Чињеница да је препознато тачно шест наслова није довољан доказ да су све границе и целокупни садржај секција исправно издвојени. Она представља структурну проверу коју треба допунити садржинском анализом.

За водич се чита PDF *outline*, односно скуп обележивача. Рекурзивним проласком кроз *outline* бележе се наслов, страница и путања предака. Затим се на наведеној страници тражи одговарајући текстуални наслов. Ако је наслов дугачак, код испробава и краће кандидате. Уколико нема употребљивих обележивача или није могуће повезати обележивач са текстуалним насловом, примењује се *fallback* подела по страницама.

За документ о интеракцијама користи се листа познатих назива, као што су *Vrste interakcija*, *Farmakokineticke interakcije*, *Resorpcija*, *Metabolizam* и *Interakcije lekova sa hranom*. Провера је усмерена на самосталне линије које имају облик наслова. Ова правила су прилагођена конкретном документу и не треба их описивати као универзално препознавање секција у свим фармаколошким приручницима.

Препознате секције представљају *parent* ниво структуре документа. У наредној фази се свака *parent* секција, ако је потребно, дели на мање *child* сегменте применом ограничења дужине и преклапања.

5.4. Parent-child сегментација

Функција *build_parent_child_documents* најпре обрађује већ формиране *parent* секције. За сваку секцију припрема се текст са *retrieval* префиксом, док се оригинално тело секције прослеђује функцији *chunk_parent_sections*. На тај начин се логички садржај секције дели на *child* сегменте пре додавања префикса сваком појединачном сегменту.

Подразумевана величина тела *child* сегмента износи 1.000 знакова, док је максимално преклапање постављено на 200 знакова. Функција *split_text* покушава да текст подели на границама пасуса, редова и речи. При спајању издвојених јединица настоји се да се очува читљив текст, док се део завршетка претходног сегмента може пренети на почетак наредног. Ефективна величина преклапања зависи од размака између јединица текста и конкретне гране извршавања. Због тога није прецизно тврдити да се свака два узастопна *child* сегмента преклапају са тачно 200 знакова.

У систему је реализована структурно вођена подела уз коришћење сопственог текстуалног *splittera*. *Parent* секције постављају границе између логичких целина, док се *child* сегменти формирају унутар тих граница.

Сваки *child* сегмент наслеђује идентитет изворног PDF документа и *parent* секције.

Додатно му се придружују редни број сегмента унутар секције и укупан број сегмената те секције. На тај начин сваки резултат претраге може да се повеже са изворним документом, *parent* секцијом и својом позицијом унутар те секције.

Retrieval префикс је део поља *page_content*, па улази и у *dense* и *sparse* репрезентацију које се формирају над текстом документа. Он остаје доступан и приликом приказа пронађеног сегмента. На тај начин назив лека, активна супстанца и други идентификациони подаци постају део текста који се користи у векторској претрази.

Последица додавања префикса је да укупан текст *child* записа може бити дужи од 1.000 знакова, иако је ограничење примењено на тело секције пре додавања префикса. У измереном корпусу најдужи *child* запис са префиксом имао је 1.366 знакова. Наведена вредност односи се на укупан текстуални садржај записа, а не само на његово подељено тело.

5.5. Припрема записа за Chroma Cloud

Функција *prepare_documents_for_chroma* прима већ формиране *Document* објекте. Њена одговорност није препознавање типа документа, избор секције или формирање *child* сегмената, већ техничка припрема записа за упис у *Chroma Cloud*. За сваки улазни текст функција проверава да ли његова UTF-8 величина премашује дозвољено ограничење. Ако је потребно, текст дели на више физичких *Cloud* записа, допуњује њихове метаподатке и враћа припремљену листу докумената.

Ограничење је постављено на 16*1024, односно 16.384 бајта по текстуалном запису. За ASCII текст број знакова и број бајтова се поклапају, али ћирилички знакови и латиничка слова са дијакритицима могу заузети више бајтова у UTF-8 кодирању.

Код најпре покушава да групише целе линије. Ако једна линија сама премашује дозвољену величину, дели је по *Unicode* знаковима, при чему се величина сваког дела одређује на основу његове UTF-8 репрезентације. На тај начин се физички записи задржавају унутар ограничења броја бајтова, а не само унутар ограничења броја знакова.

Сваки физички део добија вредност *cloud_subchunk_index*, која означава његов редни број унутар првобитног записа. Ово поље не представља нову логичку *child* сегментацију, већ само техничку поделу текста ради ограничења величине *Cloud* записа.

Када се предугачак *parent* запис подели на више физичких делова, ти делови задржавају исти *parent_id*. На основу тог идентификатора функција *format_parent_records* може да их сортира по пољу *cloud_subchunk_index* и поново повеже у целину.

5.6. Векторизација, колекција и упис

Chroma клијент добија *host*, *tenant*, *database* и API кључ из конфигурације окружења. Подразумевани назив колекције у коду је *diplomski_rag*. За ову колекцију *schema* дефинише *dense* индекс над текстом документа и *sparse* индекс под кључем *sparse_embedding*.

Листинг 5.2 приказује конфигурацију *dense embedding* функције. Инструкције су дефинисане у речнику *QWEN_RETRIEVAL_INSTRUCTIONS*. Једна инструкција намењена је опису текста документа, док је друга намењена формулисању корисничког питања за проналажење релевантних пасуса о лековима, интеракцијама и апотекарској пракси. Конкретне системске инструкције за документе и корисничке упите приказане су у листинзима 5.3 и 5.4.

Листинг 5.2. Избор *Cloud embedding* модела из *vector_store.py*

```
def qwen_embedding_function() -> ChromaCloudQwenEmbeddingFunction:
    """Dense embedding function executed by Chroma Cloud."""

    return ChromaCloudQwenEmbeddingFunction(
        model=ChromaCloudQwenEmbeddingModel.QWEN3_EMBEDDING_0p6B,
        task=DEFAULT_CHROMA_QWEN_TASK,
        instructions=QWEN_RETRIEVAL_INSTRUCTIONS,
    )
```

Листинг 5.3. Qwen инструкција за векторизацију текста документа

Represent this Serbian pharmacy document chunk for retrieval.

Листинг 5.4. Qwen инструкција за векторизацију корисничког упита

Represent this Serbian pharmacy question for retrieving relevant drug leaflet, interaction, or pharmacy practice passages.

Приликом позива *get_or_create_collection()* прослеђује се *schema* објекат који описује структуру колекције и њене индексе. У конкретном пројекту *schema* садржи *dense* индекс са *Qwen embedding* функцијом и *sparse* индекс са *SPLADE embedding* функцијом. Оба индекса користе текст из поља *DOCUMENT*, док је *sparse* индекс доступан под кључем *sparse_embedding*. Истовремено се прослеђује *embedding_function=None*. Разлог је то што су *embedding* функције већ дефинисане у конфигурацији одговарајућих индекса унутар *schema* објекта. На тај начин се избегава истовремено задавање *embedding* функције у *schema*-и и на нивоу колекције, што би могло довести до конфликта у конкретној верзији *Chroma SDK*-а.

Упис се врши серијски, у пакетима од 16 записа. Апликација *Chroma Cloud*-у доставља идентификаторе, текстове и очишћене метаподатке, док конфигуриране *Cloud embedding* функције на основу текстуалног поља *DOCUMENT* формирају *dense* и *sparse* репрезентације.

Листинг 5.5. Упис једног пакета из *ChromaVectorStore.upload_documents*

```
self.collection.upsert(
    ids=[document_id(document, start + index) for index, document in enumerate(batch)],
    documents=[document.page_content for document in batch],
    metadatas=[clean_metadata(document.metadata) for document in batch],
)
```

Функција *upsert* омогућава унос новог записа или измену постојећег записа са истим идентификатором. Ако се због промењеног редоследа обраде или измењених улазних података израчуна нови идентификатор, стари запис се тиме не уклања аутоматски. Због тога потпуно освежавање колекције и инкрементални упис нису исти поступак.

Команда са опцијом *--reset* брише изабрану *Cloud* колекцију и поново је креира, након чега у њу уписује припремљене записе. Овај поступак обезбеђује чисту изградњу колекције, али није атомска замена постојеће верзије. Ако се упис прекине, колекција може остати делимично попуњена.

5.7. Претрага и формирање *retrieval* излаза

Претрага у пројекту реализована је у класи *ChromaVectorStore*, у датотеци *vector_store.py*. Основна метода је *search()*, која прима текст корисничког упита и враћа најрелевантније *child* записе из *Chroma Cloud* колекције.

Претрага се врши над *child* записима, јер они представљају мање текстуалне

јединице погодније за прецизно проналажење релевантног садржаја. Пре покретања претраге проверава се да ли колекција садржи *child* записе. Уколико *child* записи не постоје, метода враћа празну листу и претрага се не извршава.

Након тога одређује се број кандидата који ће бити разматрани у почетној фази претраге:

$$\text{limit} = \min(\max(\text{top_k}, \text{candidate_count}), \text{child_count}) \quad (2)$$

Параметар *top_k* одређује највећи број *child* резултата у коначној листи, док *candidate_count* одређује ширину почетног скупа кандидата. Подразумевана вредност параметра *candidate_count* је 60. То значи да се у почетној фази може разматрати до 60 *child* записа, у зависности од њиховог укупног броја. Ова вредност не представља број резултата који ће бити приказани кориснику, нити број сегмената који се обавезно прослеђује генеративном моделу.

Када је укључена хибридна претрага, пројекат истовремено користи *dense* и *sparse* претрагу. *Dense* део користи подразумевани *dense* индекс, док се *sparse* део позива преко кључа *sparse_key*. За оба начина претраге користи се исти текст упита. Хибридни ранг конструише се помоћу два *Knn* израза и *Rrf* механизма, као што је приказано у листингу 5.4.

Листинг 5.6. Конструисање хибридног ранга из *vector_store.py*

```
rank = Rrf(
    ranks=[
        Knn(query=query_text, limit=limit, return_rank=True),
        Knn(query=query_text, key=self.sparse_key, limit=limit, return_rank=True),
    ],
    weights=[self.dense_weight, self.sparse_weight],
    k=60,
)
```

Први *Knn* израз односи се на *dense* претрагу, док други користи *sparse* индекс регистрован под кључем *sparse_embedding*. *Dense* и *sparse* претрага формирају засебне ранг-листе, а *Rrf* их комбинује на основу позиција резултата у тим листама. Подразумеване тежине су *dense* = 0.7 и *sparse* = 0.3. Оваквим избором тежина предност се даје *dense* претрази, која процењује семантичку сличност упита и садржаја, док *sparse* компонента доприноси проналажењу резултата на основу лексичког поклапања значајних термина.

Хибридни ранг се затим повезује са филтером за *child* записе. Иако су *parent* и *child* записи смештени у истој колекцији, примарна претрага се врши само над записима чија је вредност метаподатка *record_type* једнака *child*. На тај начин *parent* записи не учествују као самостални кандидати у активној *retrieval* путањи.

Уколико је укључено груписање по изворном документу, претрага се додатно конфигурише на следећи начин:

Листинг 5.7. Груписање по изворном документу

```
search = search.group_by(
    GroupBy(
        keys=K("source_document_id"),
        aggregate=MinK(
            keys=K.SCORE,
            k=self.group_by_document_limit,
        ),
    )
)
```

Груписање се врши на основу поља *source_document_id*, односно идентификатора изворног PDF документа. За сваки изворни документ задржава се ограничен број најбоље ранжираних *child* резултата. Подразумевана вредност је два *child* резултата по изворном документу. Овај механизам повећава разноврсност извора, али може ограничити број корисних сегмената који потичу из једног великог документа.

Хибридна претрага извршава се унутар *try* блока. Ако током њеног извршавања дође до изузетка, програм исписује упозорење и покушава *dense* претрагу као резервну путању.

Листинг 5.8. Резервна *dense* путања за хибридную претрагу

```
try:
    results = self.hybrid_search(query_text, limit, should_group)
except Exception as exc:
    print(f"[WARN] Hybrid search failed; falling back to dense query: {exc}")
    results = self.dense_search(query_text, limit)
```

Изабрани *child* резултати остају у свом изворном облику и њихов текст се користи за формирање контекста који се прослеђује генеративном моделу. Метаподатак *parent_id* и даље остаје доступан како би се *child* резултат повезао са *parent* секцијом, изворним документом и логичком структуром корпуса.

Оцена приказана уз резултат представља оцену *retrieval*-а за пронађени *child* запис. Она није ново израчуната сличност целе *parent* секције. Метаподаци као што су *retrieved_child_id*, *retrieved_child_score* и *retrieved_child_chunk_index* могу да сачувају информације о пронађеном *child* сегменту и његовој позицији. На крају се враћа највише *top_k child* резултата. Ако је укључено груписање или ако је број доступних *child* записа мањи од задатог лимита, коначна листа може садржати мање резултата од вредности *top_k*.

ChromaRetriever не учитава PDF документе, не извршава поновну сегментацију и не реконструише *parent* секције. Он прослеђује питање слоју *ChromaVectorStore* и резултате претвара у листу објеката типа *RetrievedDocument*. На тај начин *RAGPipeline* не мора да познаје интерну структуру *Chroma* одговора.

Листинг 5.9. Класа *RetrievedDocument*

```
@dataclass
class RetrievedDocument:
    text: str
    metadata: dict[str, Any]
    score: float
    document_id: str | None = None
    chroma_score: float | None = None
    distance: float | None = None
    search_type: str | None = None
```

5.8. Формирање контекста и позив *Gemini* модела

Метода *RAGPipeline.answer()* представља завршну фазу RAG тока. Њена улога је да прими питање корисника, преузме релевантне *child* сегменте преко *retriever* компоненте, формира *prompt* и, уколико је пронађен непразан контекст, проследи захтев *Gemini* моделу. Резултат обраде је објекат типа *RAGResponse*, који обједињује питање, генерисани одговор, изворе, пронађене документе и коришћени *prompt*.

У првом кораку метода *answer()* прослеђује питање *retriever* компоненти. *Retriever* враћа листу објеката типа *RetrievedDocument*. Поље *text* у тим објектима представља текст пронађеног *child* сегмента.

Након завршетка *retrieval* фазе проверава се да ли је враћена листа празна. Ако *retrieval* не врати ниједан резултат, *Gemini* модел се не позива. Систем тада директно

враћа унапред дефинисану поруку којом обавештава корисника да у доступним документима није пронађен контекст потребан за генерисање одговора.

Ако *retrieval* врати непразну листу резултата који нису довољно релевантни, не постоји посебна детерминистичка провера или праг који би такве резултате аутоматски одбацио. Они се прослеђују *Gemini* моделу, а од модела се очекује да на основу системске инструкције процени да ли је достављени *child* контекст довољан за одговор. Ако контекст не подржава одговор, модел треба да наведе да у доступним документима нема довољно информација.

Ако листа пронађених докумената није празна, формира се коначни *RAG prompt*.

Листинг 5.10. Формирање *RAG prompt*-а

```
prompt = build_rag_prompt(  
    question=question,  
    documents=retrieved_documents,  
    max_context_chars=self.max_context_chars,  
)
```

Функција *build_rag_prompt()* најпре позива функцију *format_context()*, која *retrieved* резултате претвара у текстуални блок контекста. За сваки пронађени *child* сегмент формира се посебан блок. Он садржи редни број резултата, *retrieval* оцену, ознаку извора, информације о леку и активnoj супстанци када постоје, назив секције, тип записа и текст *child* сегмента.

Након блока са контекстом додају се питање корисника и инструкција за генерисање одговора. На тај начин се у једном *prompt*-у обједињују пронађени *child* контекст, питање корисника и инструкције за генерисање одговора.

Количина текста која се укључује у *prompt* ограничена је параметром *max_context_chars*. Његова подразумевана вредност у пројекту износи 14.000 знакова. Ова вредност представља апликационо ограничење текста *retrieval* контекста, а не максималну величину контекстног прозора *Gemini* модела. Ако сви *retrieved child* сегменти не могу да стану у расположиви буџет, контекст се ограничава према редоследу резултата и расположивом броју знакова.

Поред главног *prompt*-а, *Gemini* модел добија и системску инструкцију дефинисану у датотеци *rag_prompt.py*. Системска инструкција одређује понашање модела и захтева да модел одговара на српском језику, користи достављени контекст и не измишља медицинске информације. Такође, модел треба да наведе када достављени контекст није довољан и да генерисани одговор не представља замену за савет лекара или фармацеута.

Инструкција не захтева претрагу интернета нити коришћење извора који нису прослеђени у *prompt*-у. Захтев за навођење извора формулисан је условно, изразом „када је корисно“, па модел није програмски обавезан да уз сваку тврдњу наведе формални цитат. Извори који се приказују у апликацији издвајају се из метаподатака *retrieved child* сегмената и не морају бити истоветни формалним цитатима које модел евентуално наведе у тексту одговора. Системска инструкција коришћена у апликацији је приказана у листингу 5.11.

Листинг 5.11. Основна системска инструкција коришћена у апликацији

Ti si RAG asistent za pitanja na osnovu dokumenata o lekovima.

Odgovaraj na srpskom jeziku.

Koristi samo informacije iz prosledjenog konteksta.

Ako odgovor nije podrzan kontekstom, jasno reci da nema dovoljno informacija.

Ne izmisljaj doze, indikacije, kontraindikacije ili nezeljena dejstva.

Kada je korisno, navedi izvore na kraju odgovora.

Ovo nije zamena za savet lekara ili farmaceuta.

Ако је пронађен најмање један *child* резултат, *RAGPipeline* позива *Gemini* клијент. Класа *GeminiFlashClient* овај позив прослеђује *Google Gen AI SDK*-у помоћу методе *interactions.create()*, преглед основних параметара овог позива и њиховог значења дат је у табели 5.

Листинг 5.12. Позив методе *interactions.create()*

```
interaction = self.client.interactions.create(
    model=self.model_name,
    input=prompt,
    system_instruction=system_instruction,
    generation_config={
        "max_output_tokens": self.max_output_tokens,
    },
    store=False,
)
```

Табела 5. Основни параметри методе *interactions.create()*

Параметар	Значење
<i>model</i>	Назив <i>Gemini</i> модела који се користи за генерисање одговора
<i>input</i>	Главни RAG <i>prompt</i> који садржи <i>child</i> контекст, питање и задатак
<i>system_instruction</i>	Системска инструкција која одређује начин понашања модела
<i>max_output_tokens</i>	Горња граница броја токена генерисаног одговора
<i>store=False</i>	Захтев да се интеракција не чува као трајна конверзација у оквиру овог API позива

На крају методе *answer()* формира се објекат типа *RAGResponse*. Објекат *RAGResponse* обједињује резултат генерисања и податке потребне за приказ одговора и његових извора. Поред питања и одговора, садржи пронађене сегменте са пратећим подацима и припремљени текст упита моделу, чиме се омогућава увид у контекст коришћен при обради питања. Поља овог објекта и њихов садржај приказани су у табели 6.

Листинг 5.13. Враћање објекта типа *RAGResponse*

```
return RAGResponse(
    question=question,
    answer=answer_text,
    sources=extract_sources(retrieved_documents),
    retrieved_documents=retrieved_documents,
    prompt=prompt,
)
```

Табела 6. Поља објекта *RAGResponse*

Поље	Садржај
<i>question</i>	Оригинално питање корисника
<i>answer</i>	Генерисани одговор или <i>fallback</i> порука ако нема резултата
<i>sources</i>	Извори издвојени из метаподатака <i>retrieved child</i> сегмената
<i>retrieved_documents</i>	<i>Child</i> документи које је вратио <i>retriever</i> , заједно са њиховим метаподацима и оценама
<i>prompt</i>	Коначни <i>prompt</i> прослеђен Gemini моделу

Оваква организација раздваја *retrieval child* сегмената, формирање *prompt*-а, генерацију одговора и приказ извора. Истовремено, систем детерминистички проверава само да ли је *retrieval* вратио резултате. Процена да ли је непразан контекст заиста довољан за одговор препуштена је генеративном моделу, у складу са системском инструкцијом.

5.9. Кориснички интерфејс и историја

Streamlit апликација има главни екран за постављање питања и приказ генерисаних одговора. Бочна трака омогућава избор колекције, броја коначних извора, броја почетних кандидата, ограничења броја *child* резултата по PDF документу, величине контекста и максималног броја излазних токена. Доступни прекидачи укључују хибридную претрагу, груписање резултата и приказ *retrieved* контекста.

Испод одговора налази се табела са називом лека, активном супстанцом, PDF-ом, секцијом, страницом, типом и оценом. Прошириви приказ омогућава читање текста и сажетих метаподатака. Нема уграђеног PDF прегледача са истицањем конкретног пасуса, извоза разговора или администрације корпуса кроз UI.

Стање разговора чува се у *st.session_state*. *Streamlit* сесијско стање омогућава да подаци остану сачувани током поновног извршавање скрипте у истој сесији; не представља само по себи трајну базу разговора [23]. Нова сесија или рестарт сервера не реконструишу претходну историју из диска.

Искључивање опције за контекст мења видљивост, не брише сачуване податке. Дугме за чишћење разговора уклања све поруке и њихове доказе из сесије. Ако *backend* тренутно није доступан, стари одговори остају видљиви, а нови унос је онемогућен.

На сликама 5.3 и 5.4 приказан је стварни интерфејс са примером питања и одговора.

Podešavanja

Chroma kolekcija

diplomski_rag

Broj izvora

5

Kandidati za pretragu

60

Rezultati po PDF-u

2

Konekt za Gemini

14000

Dužina odgovora

1200

Hybrid search

Grupisi po PDF-u

Prikaži kontekst

Očisti razgovor

Deploy

Apotekarski RAG asistent

Sistem je namenjen kao pomoc pri radu u apoteci i ne zamenjuje stručnu procenu farmaceuta ili lekara.

Šta su farmakokinetičke interakcije lekova?

Farmakokinetičke interakcije lekova se mogu javiti kod svih farmakokinetičkih procesa. Nastaju kada jedan lek utiče na resorpciju, distribuciju, metabolizam i/ili izlučivanje (ekskreciju) drugog leka.

Iako se ove interakcije mogu predvideti, zbog velike interindividualne varijabilnosti ne može se tačno predvideti u kojoj meri će biti od kliničkog značaja.

Glavni mehanizmi kroz koje se ispoljavaju su:

- **Resorpcija:** Najčešće dolazi do smanjenja stepena resorpcije leka (usled promena u pH vrednosti gastrointestinalnog trakta, pokretljivosti GIT-a, vezivanja lekova u komplekse, uticaja hrane, bakterijske flore ili indukcije/inhibicije P-glikoproteina).
- **Distribucija:** Dolazi do kompeticije lekova za vezivanje za proteine plazme (posebno značajno kod lekova sa visokim stepenom vezivanja za proteine plazme).
- **Metabolizam:** Predstavljaju najznačajnije interakcije, a nastaju usled indukcije ili inhibicije izoenzima CYP450.
- **Ekskrecija (izlučivanje):** Promene u bžrini glomerularne filtracije, tubularne sekrecije, tubularne reapsorpcije ili pH vrednosti i protoka urina utiču na smanjenje ili povećanje izlučivanja lekova putem bubrega.

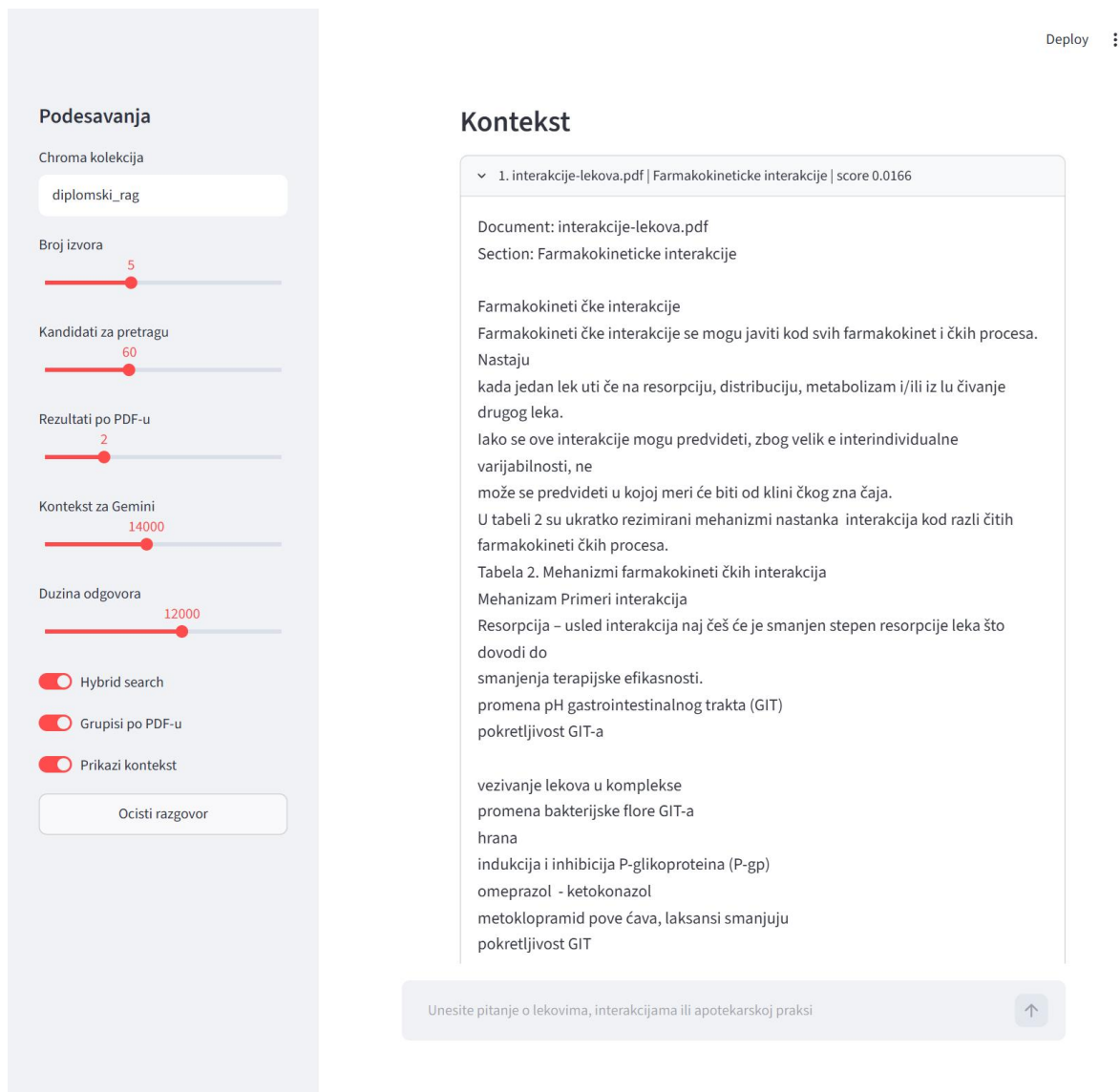
Izvor:

- [interakcije-lekova.pdf](#) , strana 6, sekcija *Farmakokinetičke interakcije*

Unesite pitanje o lekovima, interakcijama ili apotekarskoj praksi

Слика 5.3. Питање и одговор у Streamlit интерфејсу.

35



Слика 5.4. Почетак пронађене секције у интерфејсу; приказ није једнак ограниченом контексту послатом моделу.

6. Тестирање и експериментални резултати

6.1. Софтверски тестови

При локалном извршавању успешно је завршено свих 37 тестова, без пријављених неуспеха. Подела по датотекама приказана је у табели 7. Део тестова користи синтетичке документе и замене зависности, док део чита стварне локалне PDF датотеке. Успешан исход потврђује тестиране функционалности и својства, али није доказ потпуне покривености кода или клиничке исправности одговора.

Табела 7. Расподела аутоматизованих тестова

Датотека	Број	Главни предмет провере
<i>test_data_loader.py</i>	7	Типови докумената, странице, метаподаци, наставак после неисправног PDF-а
<i>test_document_sections.py</i>	7	Ћирилица, шест секција, границе секција, parent-child везе, outline водича
<i>test_retrieval_evaluation.py</i>	6	Метрике, подразумевани child-only режим и бележење изабраног режима
<i>test_retriever_rag.py</i>	4	Делегирање child претраге, child текст у prompt-у и изостанак LLM позива без контекста
<i>test_vector_store.py</i>	7	Ограничење величине, метаподаци, поновљив ID за исти улаз
<i>test_ui_history.py</i>	6	Историја одговора, извори, контекст, грешке и чишћење сесије

Тестови обраде секција проверавају да *child* сегмент не садржи текст две *parent* секције и да се веза са *parent* секцијом задржава у метаподацима. Тест над упутством за лек проверава присуство шест главних секција.

За тестирање корисничког интерфејса коришћен је *Streamlit AppTest* омогућава програмско покретање апликације и испитивање њених елемената и стања [24]. У тестовима овог пројекта RAG зависност је замењена контролисаним одговором. Тако се изолује понашање UI-ја од доступности *Cloud* сервиса.

Додатно се проверава да промена приказа контекста не покреће поново генерисање, да стари одговор задржава сопствену копију метаподатака и да недоступност *backend*-а не скрива већ сачувану историју. Провера није визуелна оцена изгледа на различитим ширинама екрана, нити доказ трајног чувања разговора. У тестном запису постоје упозорења о застарелом *Streamlit* параметру *use_container_width*; она нису изазвала пад тестова, али представљају посао за одржавање.

6.2. Провера корпуса и сегментације

Поновљена локална анализа обухватила је 31 PDF документ и 339 страница са успешно издвојеним текстом. У корпусу је препознато 29 упутстава за лекове, један водич добре апотекарске праксе и један документ о интеракцијама лекова. Формиране су 493 логичке *parent* секције и 1.361 *child* сегмент. Код свих 29 упутстава препознато

је шест главних секција и уводни део. На тај начин упутства за лекове дају 203 *parent* секције, водич 279, а документ о интеракцијама 11 *parent* секција.

Није пронађен *child* сегмент без одговарајућег *parent_id*, нити запис коме недостаје обавезни *retrieval* префикс или неки од проверених обавезних метаподатака. Ове провере потврђују структурну конзистентност локално формираних *Document* објеката пре припреме за *Cloud* упис.

Две *parent* секције премашују ограничење од 16 KiB по текстуалном *Cloud* запису. Због тога се приликом припреме за *Chroma Cloud* деле на више физичких делова. Тако се од 493 логичке *parent* секције добија 495 физичких *parent Cloud* записа, док број *child* сегмената остаје 1.361. Укупан број припремљених физичких записа зато износи 1.856.

Разлика између 493 логичке *parent* секције и 495 физичких *parent* записа не представља дуплирање секција нити додатну логичку сегментацију. Она је последица техничког ограничења величине појединачног *Cloud* записа. Физички делови задржавају исти идентификатор *parent* секције и разликују се помоћу поља *cloud_subchunk_index*.

Архивирани *Cloud* инвентар, добијен пагинираним читањем колекције, бележи исти укупан број од 1.856 физичких записа. *Parent* делови су сачувани у колекцији и векторизовани. *Parent* физички делови служе за очување структуре и праћење порекла, а не као текстуална замена за *child* контекст који се прослеђује *Gemini* моделу.

6.3. Методологија евалуације претраге

За евалуацију претраге припремљена су два скупа питања: основни и допунски. Основни скуп садржи 15 питања, од којих се десет односи на лекове, два на интеракције лекова и три на водич добре апотекарске праксе. Намењен је поређењу конфигурација претраге према успешности проналажења унапред одређених докумената и секција. Допунски скуп садржи четири питања и служи за испитивање одабраних случајева: постављања питања на ћирилици и енглеском језику, као и проналажења конкретних информација на основу кратких, разговорно формулисаних упита. Код три допунска питања захтева се и присуство одређеног израза у враћеном тексту, чиме се провера не своди само на поклапање документа и секције. Резултати основног и допунског скупа разматрају се одвојено, јер основни скуп служи за упоредну оцену претраге, док допунски омогућава анализу појединачних изазовних случајева.

Прецизност и одзив представљају стандардне мере успешности система за проналажење информација. Код ранжираних резултата мерење на првих k места усмерава пажњу на резултате које корисник или наредна компонента заиста добија [25]. У овом пројекту начин израчунавања мера одређена је функцијама које пореде очекивања са метаподацима и текстом враћених *child* резултата. Због тога је важно нагласити да ове метрике не мере непосредно потпуност конкретног доказног пасуса, већ пре свега успешност проналажења очекиваног документа или секције.

Нека је A_q скуп очекиваних услова за питање q , а L_q^k првих највише k враћених резултата. Нека $M(r, a)$ означава да резултат r задовољава очекивање a . Тада евалуациони код рачуна:

$$P@k(q) = \frac{\sum_{r \in L_q^k} I[\exists a \in A_q: M(r, a)]}{k} \quad (4)$$

$$R@k(q) = \frac{|\{a \in A_q: \exists r \in L_q^k: M(r, a)\}|}{|A_q|} \quad (5)$$

У формули (4) именилац остаје k и када је враћено мање резултата. У формули (5) броје се различита испуњена очекивања, а не сва понављања исте релевантне секције. На пример, ако једно очекивање има један одговарајући резултат међу првих пет, добијају се вредности $P@5=0,2$ и $R@5=1$. Ако свих пет *child* резултата задовољава исто очекивање, обе мере имају вредност 1. У том случају прецизност расте због већег броја релевантних појављивања, али се број различитих покривених циљева не повећава.

$\text{Hit}@k$ има вредност један ако постоји бар једно поклапање у првих k резултата, а нула у супротном. Реципрочни ранг за једно питање износи $1/j$, где је j позиција првог поклапања, или нула ако поклапање не постоји у првих k места. Просек тих вредности представља $\text{MRR}@k$. И остале мере су макроусредњене, па свако питање има једнаку тежину.

Наведене формуле не користе нумеричку вредност *dense* или *RRF* оцене, већ редослед резултата и испуњеност очекивања. Због тога се њима могу поредити различите врсте претраге чак и када њихови сирови скорови имају различите скале.

За испитивање утицаја начина рангирања и груписања резултата упоређене су три конфигурације претраге, приказане у табели 8. Конфигурација H-G користи хибридную претрагу, која комбинује *dense* и *sparse* репрезентацију методом *RRF*, уз ограничење на највише два резултата из истог PDF документа. Конфигурација H-N користи исто хибридно рангирање, али без груписања по изворном документу. Конфигурација D-N користи само *dense* репрезентацију, такође без груписања. Све конфигурације испитују се над истим корпусом, питањима и очекиваним резултатима, а за свако питање оцењује се највише пет враћених сегмената.

Табела 8. Конфигурације експеримента претраге

Ознака	Рангирање	Остали заједнички услови
H-G	Qwen + SPLADE, RRF	60 кандидата, до пет резултата
H-N	Qwen + SPLADE, RRF	Исти корпус, упити и очекивања
D-N	Qwen	Исти корпус, упити и очекивања

Обе хибридне варијанте користе *dense* и *sparse* тежине 0,7 и 0,3, као и *RRF* параметар $k=60$. У H-G конфигурацији груписање ограничава број најбоље ранжираних *child* резултата на највише два по изворном PDF документу. Хибридна претрага ни у једном забележеном случају није прешла на резервну *dense* претрагу. Поређење H-G са H-N мери утицај груписања, а поређење H-N са D-N утицај различитог рангирања при истим осталим условима.

6.4. Резултати основног скупа

Резултати у табели 9 показују да подразумевана варијанта претраге H-G има $\text{Hit}@1 = 0,800$, што значи да је први резултат био релевантан за 12 од 15 питања. На првих пет места најмање једно очекивање испуњено је за 13 питања. Код два питања о активној супстанци очекују се по две секције из различитих PDF докумената, због чега $\text{Recall}@1$ није једнак $\text{Hit}@1$.

Табела 9. Макроусредњени резултати над 15 питања

Варијанта	k	Precision@k	Recall@k	Hit@k	MRR@k
H-G	1	0,800	0,733	0,800	0,800
H-G	3	0,533	0,867	0,867	0,833
H-G	5	0,347	0,867	0,867	0,833
H-N	1	0,800	0,733	0,800	0,800
H-N	3	0,667	0,833	0,867	0,833
H-N	5	0,613	0,900	0,933	0,850
D-N	1	0,733	0,700	0,733	0,733
D-N	3	0,600	0,800	0,867	0,789
D-N	5	0,480	0,933	0,867	0,789

Искључивање груписања у хибридној претрази повећало је Recall@5 са 0,867 на 0,900, док је Hit@5 повећан са 13/15 на 14/15 питања. Ова разлика не означава једноставно једно додатно потпуно покривено питање, јер се очекивања не односе увек на само један документ. На пример, конфигурација H-N проналази очекивану секцију о складиштењу у водичу, али код питања о нежељеним дејствима *амлодипина* покрива само један од два очекивана PDF документа. Свих пет резултата за то питање потиче из секције о нежељеним дејствима документа *amlopin-combo.pdf*, док очекивани документ *norvasc.pdf* није пронађен. Због тога су истовремено могуће вредности Precision@5 = 1 и Recall@5 = 0,5.

У овом основном скупу H-N има веће вредности Recall@5 и MRR@5 од D-N конфигурације: 0,900 према 0,833 и 0,850 према 0,789. Ова разлика односи се на конкретна испитана питања и не представља доказ опште надмоћи хибридне претраге. H-G не покрива два очекивања која се односе на Водич, H-N не покрива питање о издавању лекова и део очекивања за амлодипин, док D-N не покрива оба наведена питања о Водичу и такође не покрива један део очекивања за амлодипин. Сам висок score појединачног child резултата не може да надокнади непокривено очекивање.

Виша вредност Precision@5 код негруписаних варијанти делом је повезана са тим што се у резултатима задржава више *child* сегмената из исте означене секције. Груписање ограничава понављање резултата из истог PDF документа, али истовремено може искључити различите пасусе истог извора који су потребни за потпун одговор. Због тога утицај груписања треба процењивати заједно са покривеношћу различитих очекивања и садржајем контекста, а не само на основу једног агрегатног показатеља.

Код H-G конфигурације одзив се не повећава између три и пет резултата. Додатна два места зато снижавају прецизност, без повећања покривености означених циљева. То не значи да је k=3 универзално оптимална вредност, јер скуп садржи претежно по једно очекивање по питању и није потпуно исцрпан. Ужи закључак је да повећање броја враћених *child* резултата у овом мерењу није исправило два преостала промашаја конфигурације H-G.

Раздвајање резултата по врсти документа показује неравномерност успешности. За десет питања о лековима H-G има Recall@5 = 1,000 и MRR@5 = 0,950; за два питања о интеракцијама обе мере имају вредност 1,000; док за три питања о водичу обе мере износе 0,333. Овај резултат указује на значај прилагођене обраде различитих типова докумената, али не доказује да је препознавање структуре једини узрок разлике. Групе

се разликују и по формулацији питања, броју очекиваних секција и обиму докумената.

Скуп питања о лековима обухвата осам различитих PDF докумената, а не свих 29 докумената у корпусу. Формулације питања често су блиске називима стандардних секција. Зато савршен одзив Н-Г конфигурације на овом подскупу не треба представљати као потпуну проверу целе базе лекова, свих могућих корисничких питања или потпуности одговора о нежељеним дејствима. Проналажење једног *child* сегмента из исправне секције не значи да су пронађене све информације које се налазе у тој секцији.

6.5. Додатне језичке провере

Допунски скуп има четири питања. Проверава се да ли је одговарајућа секција пронађена.

За ћирилични упит хибридна варијанта међу првим резултатима враћа секције других лекова, док густа варијанта без груписања поставља на прво место. Ово је конкретан контрапример тврдњи да мултијезични густо модел аутоматски обезбеђује једнак квалитет целог хибридног система на оба писма. SPLADE модел је означен као енглески, али један пример није довољан за доказ да је он једини узрок грешке.

Агрегатни Hit@5 износи 0,750 за Н-Г и 1,000 за D-Н. Пошто су истовремено промењени рангирање и груписање, ово поређење не изолује њихове појединачне утицаје. За то би требало додати Н-Н на истом допунском скупу и увести више латиничних, ћириличних и енглеских парафраза истог питања. Четири случаја служе за откривање проблема, не за општу оцену језичке покривености.

6.6. Провера генерисања и времена одзива

Провера целокупног RAG тока обухвата пет позива генеративном моделу *gemini-3.6-flash*. Свих пет позива вратило је непразан текстуални одговор. Табела 10 приказује посматрано понашање система и измерено трајање позива, без представљања ових примера као клинички оцењеног скупа.

Табела 10. Квалитативна провера целог RAG тока

Случај	Посматрано понашање	Трајање позива (s)
Нежељена дејства лека Norvasc	Одговор организује информације и наводи пронађени извор	29,77
Чување лека према водичу	Формиран је одговор повезан са пронађеним <i>child</i> контекстом	41,53
Фармакокинетичке интеракције	Појам и механизми објашњени су на основу достављеног <i>child</i> контекста	28,01
Измишљени назив Zorvexalin-TEST-999	Наведено је да документи не садрже тражене информације	11,47
Питање о фудбалу изван домена система	Наведено је да достављени контекст не омогућава одговор	38,45

Последња два случаја показују корисно понашање генеративног модела: модел није попунио недостатак доказа произвољним одговором. Међутим, *retriever* је и у тим случајевима вратио *child* текстове из базе. Одбијање одговора зато је настало у фази

генерисања, на основу системске инструкције и процене модела, а не као резултат детерминистичке провере релевантности која би спречила позив генеративном моделу.

Ова чињеница представља ограничење система. Програм детерминистички прескаче позив *Gemini* модела само када *retrieval* не врати ниједан резултат. Ако *retrieval* врати непразну листу *child* сегмената, чак и када они нису довољно релевантни, они се прослеђују моделу, који затим процењује да ли достављени контекст подржава одговор.

Два успешна одбијања нису довољна за израчунавање поуздане стопе спречених халуцинација. Није спроведена независна оцена фармацеута, систематска провера сваке генерисане тврдње према изворном *child* сегменту нити поређење са ручно написаним референтним одговорима. Због тога се наведени резултати могу описати као функционална и квалитативна провера целокупног RAG тока.

6.7. Анализа грешака и ваљаности закључака

Анализа је показала да део промашаја у *retrieval*-у потиче од непотпуне анотације референтног скупа, док у другим случајевима пронађени назив или секција нису праћени довољно садржајним *child* текстом. Поред тога, ограничење *prompt*-а од 14.000 знакова може довести до тога да раније рангирани *child* сегменти заузму расположиви простор и потисну касније релевантне резултате или њихове делове. Због тога се у листи извора могу појавити документи чији *child* текст није у целости ушао у коначни *prompt*, или чији је текст у *prompt*-у само делимично присутан.

Додатне провере показале су да идентификатор записа зависи од индекса уписа, па његова стабилност важи само када су сви улазни параметри и редослед обраде непромењени. Уочено је и да *chunking* у појединим случајевима може пресећи реч, што значи да подела поштује границе *parent* секција, али не гарантује увек очување границе речи или реченице унутар *child* сегмента. Новија верзија корисничког интерфејса исправно чува изворе и контекст у историји сесије, али то не представља трајну меморију нити меморију коју генеративни модел аутоматски користи у наредним питањима.

Резултате треба тумачити у оквиру ограничења коришћеног корпуса, ограниченог броја питања, непостојања независних анотатора и могуће непотпуности скупа питања. У актуелној евалуацији враћају се *child* резултати, али се релевантност у великој мери оцењује на основу метаподатака о изворном документу и *parent* секцији. Због тога коришћене метрике не мере нужно да ли је пронађен најбољи доказни пасус нити да ли један *child* сегмент садржи све информације потребне за потпун одговор. Поред тога, релевантност код комбинованих лекова не треба аутоматски приписати само једној активној супстанци.

Ови налази не доводе у питање функционалност система, већ показују да је реч о провереном прототипу са мерљивим резултатима на ограниченом скупу питања. За поузданије закључке потребни су потпунији и прецизније аотирани референтни скупови, независна стручна валидација, јасно означени доказни *child* пасуси и очување тачно употребљеног контекста у коначном *prompt*-у.

7. Безбедност, ограничења и даљи развој

7.1. Стручна одговорност и коришћење извора

Намена развијеног система је подршка проналажењу и тумачењу фармацеутске документације, а не самостално прописивање терапије, постављање дијагнозе или замена стручног рада фармацеута. Ово ограничење произилази из природе RAG система: коначни одговор зависи од успешно пронађеног контекста, његове релевантности и начина на који га генеративни модел тумачи. Систем зато може да врати нерелевантан или непотпун сегмент, изостави услов важан за тумачење информације или генерише тврдњу која није у потпуности поткрепљена достављеним изворима.

У циљу смањења наведених ризика, системска инструкција захтева ослањање на достављени контекст, забрањује измишљање информација и подстиче модел да укаже на недостатак доказа када одговор није могуће поуздано извести из доступног материјала. Додатно, генеративни модел се не позива у случају када *retrieval* не врати ниједан резултат. Кориснички интерфејс приказује пронађене *child* сегменте, њихове метаподатке и изворне PDF документе, чиме се омогућава додатна провера генерисаног одговора. Ови механизми повећавају проверљивост система, али сами по себи не гарантују тачност генерисаног одговора.

При процени поузданости потребно је разликовати постојање извора, релевантност пронађеног сегмента и поткрепљеност конкретне тврдње. Проналажење одговарајућег документа не значи нужно да је пронађен пасус који садржи информацију потребну за одговор. Слично томе, и када је релевантан *child* сегмент прослеђен генеративном моделу, постоји могућност да информација буде непотпуно пренета или погрешно протумачена. Због тога се резултат система не треба посматрати само на основу назива приказаног извора, већ у односу на конкретан текст који је коришћен за генерисање одговора.

Посебну пажњу захтевају питања која се односе на активне супстанце присутне у више различитих препарата. Проналажење документа који садржи назив активне супстанце не значи да се све информације из тог документа могу приписати тој супстанци независно од конкретног препарата. При тумачењу таквих резултата потребно је узети у обзир назив лека, његов састав и контекст у којем је информација наведена. Метаподаци коришћени у овом прототипу представљају организациону основу за идентификацију докумената и не представљају независно проверену фармаколошку нормализацију.

Такође, различите врсте фармацеутске документације не могу се нужно обрађивати истим структурним правилима. У овом раду шест главних секција препознатих у *parser*-у односе се на структуру упутства за пацијента. Проширивање корпуса на друге регулаторне документе захтевало би увођење одговарајућих типова докумената и правила за препознавање њихове структуре, како би се избегло погрешно тумачење или губитак структурних информација.

Из наведених разлога, генерисани одговор треба посматрати као помоћни резултат претраге и читати га заједно са приказаним изворним сегментом и оригиналним документом. Клиничке и терапијске одлуке морају остати у домену квалификованог стручњака. У овом раду нису доказани усаглашеност система са захтевима који се примењују на медицински софтвер, клиничка ефикасност нити подобност за аутономно доношење одлука. Доказивање таквих својстава захтевало би шири обим испитивања, клиничку валидацију и одговарајућу стручну процену.

7.2. Преостала техничка ограничења

Пројекат има неколико техничких ограничења која произилазе из начина учитавања докумената, формирања *child* сегмената, претраге и генерисања одговора.

На улазу се користи текстуални слој PDF документа. Због тога обрада зависи од тога да ли је текст у PDF-у доступан за издвајање. Скениране странице, сложене табеле и документи са више колона могу довести до непотпуно издвојеног или неправилно распоређеног текста. Правила за препознавање секција прилагођена су типовима докумената који се користе у пројекту и не представљају универзални *parser* за сва могућа упутства о лековима.

Величина текста контролише се на два различита начина. *Child* сегменти ограничавају се бројем знакова, док се записи намењени *Chroma Cloud*-у додатно ограничавају бројем UTF-8 бајтова. Ова ограничења не представљају директну контролу броја токена које користе *embedding* модели. Пошто се *embedding* обрада извршава у *Chroma Cloud*-у, величина *child* сегмента у локалном коду сама по себи не гарантује да ће сваки запис бити обрађен без интерног скраћивања у удаљеном сервису.

У *Chroma* колекцију уписују се и *parent* и *child* записи. Оваква организација поједностављује управљање записима, али значи да се и *parent* текстови векторизују и чувају иако нису директни кандидати у примарној *retrieval* претрази.

Поновни упис у базу уз опцију *--reset* подразумева брисање изабране колекције и њено поновно креирање. Ако дође до прекида током новог уписа, колекција може остати само делимично попуњена. Поступак зато не представља атомску замену старе верзије базе новом верзијом. Поред тога, стабилност идентификатора зависи од вредности које се користе при њиховом формирању, укључујући индекс записа, па промена улазних параметара може довести до различитих идентификатора.

Retrieval компонента нема посебан модул за препознавање назива лека у питању, контролисани речник синонима, специјализовани филтер по препарату или *cross-encoder* за додатно рерангирање. Релевантност се одређује *dense* и *sparse* претрагом, уз могуће RRF комбиновање резултата. Груписање по изворном PDF документу ограничава број *child* резултата из једног извора, али не гарантује да ће сваки враћени резултат припадати различитој *parent* секцији. Због тога коначна листа може садржати мање различитих секција од очекиваног броја.

Прелазак са хибридне на *dense* претрагу у случају грешке повећава отпорност система на проблеме у хибридној путањи. Ипак, пошто се у тој грани обухвата шири тип изузетка, стварни узрок проблема може остати неразјашњен ако се исписано упозорење додатно не анализира.

У генеративном делу система постоји ограничење у начину избора текста који улази у *prompt*. Контекст се попуњава редоследом *retrieval child* резултата и ограничава на задати број знакова. Ако су раније ранжирани *child* сегменти дугачки, они могу заузети расположиви простор и потиснути касније резултате или њихове делове. Систем зато не гарантује да ће сви релевантни *child* сегменти бити укључени у коначни *prompt*, чак и када се налазе у истој *parent* секцији или изворном документу.

Историја разговора у корисничком интерфејсу служи за приказ претходних питања, одговора, извора и контекста. Она не представља трајну меморију која се користи за аутоматско проширивање наредног питања. Због тога питања која зависе од претходног одговора морају бити формулисана тако да садрже довољно самосталног контекста.

Наведена ограничења не спречавају коришћење система за претрагу и генерисање одговора на основу фармацеутских докумената. Она, међутим, ограничавају степен аутоматизације, контроле извора, преносивости и поузданости који

се може очекивати без додатне обраде докумената, напреднијег рерангирања, прецизнијег управљања контекстом и формалне провере генерисаних тврдњи.

7.3. Приоритети за унапређење

Непосредно прослеђивање текста изабраних *child* сегмената већ је реализовано у актуелној варијанти система; наредни корак је испитивање контролисаног додавања суседних пасуса када изабрани *child* сегмент не садржи све услове потребне за одговор. При томе би пронађени *child* пасус морао да остане очуван као примарни доказ, уз контролу дужине контекста и преклапања информација. Такво решење представљало би селективно проширивање контекста, а не аутоматску замену *child* сегмента целокупном *parent* секцијом. Из истог коначног списка блокова који се стварно прослеђују моделу требало би формирати и изворе и приказ у корисничком интерфејсу.

Унапређење процеса обраде докумената. Преклапање *child* сегмената треба бирати на граници речи или реченице, без произвољног одсецања завршетка. За веома дугачке листе и табеле потребан је посебан поступак који очувава наслов, колоне, јединице и услове. Пре замене целог *parser*-а било би рационално формирати визуелно проверен подскуп проблематичних PDF страница и измерити да ли ново решење заиста боље очува њихов садржај.

Побољшање претраге уз контролисане експерименте. На истим питањима, над истом верзијом корпуса и уз исту конфигурацију индекса, требало би упоредити *dense* и хибридную претрагу, различите тежине, правила избора RRF кандидата, груписање по PDF документу или *parent* секцији и евентуални *reranker*.

Додатни правац развоја представља увођење агентског RAG приступа, у којем би језички модел, поред генерисања одговора, усмеравао поступак проналажења информација. За разлику од унапред задатог тока са једном претрагом, модел би могао да разложи сложено питање на више потпитања, преформулише упит и покрене додатну претрагу када пронађени контекст није довољан. На тај начин омогућило би се повезивање информација из више докумената и постепено прикупљање контекста потребног за одговор. Овакав приступ захтевао би ограничење броја претрага и позива модела, бележење предузетих корака и јасне услове за завршетак поступка. Његову оправданост требало би проверити поређењем са постојећим секвенцијалним приступом, уз оцену поткрепљености одговора, времена извршавања и трошкова.

Као могућност даљег унапређења може се размотрити увођење трајне историје разговора која би, поред приказа претходних питања и одговора, могла да се користи и као додатни контекст приликом генерисања наредних одговора. На тај начин би систем могао да обради и упите који се ослањају на претходни ток разговора, као што су допунска питања или захтеви за појашњење већ добијене информације, без потребе да корисник поново наводи цео контекст.

8. Закључак

Реализацијом прототипа остварен је основни циљ рада: обрада докумената, мултијезична векторизација, *ChromaDB* претрага и генерисање одговора повезани су у јединствен систем за подршку раду у апотеци. Практични допринос огледа се у прилагођавању обраде различитим типовима докумената, очувању везе пронађених сегмената са изворима и омогућавању да се одговор провери увидом у коришћени текст. Модуларна организација и поступак евалуације пружају основу за испитивање појединачних компоненти и даље унапређивање система.

Корисност оваквог система огледа се у обједињеном приступу документацији и могућности да корисник постави питање природним језиком, добије сажет одговор и прегледа релевантне делове извора. Тиме се може олакшати сналажење у већем броју докумената и смањити потреба за њиховим појединачним претраживањем. Ипак, стварну уштеду времена и корист у свакодневном раду потребно је потврдити испитивањем са фармацеутима. Реализовани систем зато представља основу за помоћни информациони алат чија је улога да подржи приступ информацијама и њихову проверу, док тумачење података и доношење стручних одлука остају одговорност фармацеута.

Литература

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel и D. Kiela, „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks“, *Advances in Neural Information Processing Systems*, 2020. Досушно на: <https://arxiv.org/abs/2005.11401>.
- [2] Пројектни репозиторијум дипломски, „Изворни код система за подршку раду у апотеци“, верзија пакета 0.1.0. Изворни код (../src/diplomski/).
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser и I. Polosukhin, „Attention Is All You Need“, *Advances in Neural Information Processing Systems*, 2017. Досушно на: <https://arxiv.org/abs/1706.03762>.
- [4] N. Reimers и I. Gurevych, „Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks“, *Proceedings of EMNLP-IJCNLP*, 2019. Досушно на: <https://arxiv.org/abs/1908.10084>.
- [5] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen и W.-t. Yih, „Dense Passage Retrieval for Open-Domain Question Answering“, *Proceedings of EMNLP*, 2020. Досушно на: <https://arxiv.org/abs/2004.04906>.
- [6] T. Formal, B. Piwowarski и S. Clinchant, „SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking“, *Proceedings of SIGIR*, 2021. Досушно на: <https://arxiv.org/abs/2107.05720>.
- [7] G. V. Cormack, C. L. A. Clarke и S. Büttcher, „Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods“, *Proceedings of SIGIR*, 2009, doi: 10.1145/1571941.1572114. Досушно на: <https://cormack.uwaterloo.ca/cormacksigir09-rf.pdf>.
- [8] Qwen, „Qwen3-Embedding-0.6B: Model Card“, Hugging Face. Досушно на: <https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>.
- [9] Chroma, „Chroma Cloud Qwen“, документација интеграције векторизације. Досушно на: <https://docs.trychroma.com/integrations/embedding-models/chroma-cloud-qwen>.
- [10] Chroma, „Chroma Cloud Splade“, документација интеграције ретких репрезентација. Досушно на: <https://docs.trychroma.com/integrations/embedding-models/chroma-cloud-splade>.
- [11] Chroma, „Manage Collections“, документација. Досушно на: <https://docs.trychroma.com/docs/collections/manage-collections>.
- [12] Chroma, „Sparse Vector Search“, документација Cloud шеме. Досушно на: <https://docs.trychroma.com/cloud/schema/sparse-vector-search>.
- [13] Chroma, „Hybrid Search“, документација Search API-ја. Досушно на: <https://docs.trychroma.com/cloud/search-api/hybrid-search>.
- [14] Chroma, „Group By“, документација Search API-ја. Досушно на: <https://docs.trychroma.com/cloud/search-api/group-by>.
- [15] Chroma, „Chunking“, водич за изградњу система. Досушно на: <https://docs.trychroma.com/guides/build/chunking>.
- [16] Google, „Gemini 3.6 Flash“, документација модела Gemini API. Досушно на: <https://ai.google.dev/gemini-api/docs/models/gemini-3.6-flash>.

- [17] European Medicines Agency, „QRD Product-information Annotated Template, English, Version 10.4“. Досупно на: https://www.ema.europa.eu/en/documents/template-form/qrd-product-information-annotated-template-english-version-104_en.pdf.
- [18] Агенција за лекове и медицинска средства Србије (АЛИМС), „Pretraživanje humanih lekova – PIS“. Досупно на: <https://www.alims.gov.rs/humani-lekovi/pretrazivanje-humanih-lekova/?a=p>.
- [19] Министарство здравља Републике Србије, водич добре апотекарске праксе, 2021. Досупно на: https://www.zdravlje.gov.rs/view_file.php?cache=sr&file_id=2168.
- [20] Фармацеутска комора Србије, интеракције лекова. Досупно на: <https://www.farmkom.rs/pdf/stranice/interakcije-lekova.pdf>.
- [21] LangChain, „Document Loaders“, документација Python интеграција. Досупно на: https://docs.langchain.com/oss/python/integrations/document_loaders.
- [22] pypdf, „Extract Text from a PDF“, документација. Досупно на: <https://pypdf.readthedocs.io/en/stable/user/extract-text.html>.
- [23] Streamlit, „Session State“, документација API-ја. Досупно на: https://docs.streamlit.io/develop/api-reference/caching-and-state/st.session_state.
- [24] Streamlit, „st.testing.v1.AppTest“, документација тестирања апликација. Досупно на: <https://docs.streamlit.io/develop/api-reference/app-testing/st.testing.v1.apptest>.
- [25] C. D. Manning, P. Raghavan и H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008, одељак „Evaluation of ranked retrieval results“. Досупно на: <https://nlp.stanford.edu/IR-book/html/htmledition/evaluation-of-ranked-retrieval-results-1.html>

